Choir: State and Vision

An owner's report on the codebase, the last week, the next month, and the idea behind it. 14 October 2026. Repository state: `origin/main` at 7173cc4a0; staging (choir.news) reporting build dd47dd84, deployed 14:16 UTC. Revised about 15:30 UTC: a new section 8 on the business model and the medium-to-long-term vision (owner, about 15:05 UTC), and today's claims, search-reuse, embedding and VM density designs folded in; staging then reported build bdc99643 (deployed 14:57 UTC).

How to read this report

Choir is a large idea and a fast-moving codebase. To keep the two apart, every claim about a product element carries one of four labels:

Label	Meaning
Built, verified on staging	Code is on `main`, deployed to choir.news, and an acceptance proof or evidence record shows it working there
Built / partly built	Code exists, but staging proof is missing, partial, or stale
Designed	A design note, goal file or research document exists; little or no code
Vision	Stated by the owner or in the vision documents; no design yet

Claims about the code cite repository paths; claims about progress cite commit SHAs (short form). Section 2 explains the vision; section 3 the architecture; section 4 the principles; sections 5 and 6 the current state and the last week; section 7 the next week and month; section 8 the business model and the long-term vision; section 9 the risks. A glossary, station index and file map are in the appendices.

Product names follow the house rule: Autopaper, Autoputer, Autoradio. "World Wire", "Universal Wire" and "sourcecycled" are retired names for Autopaper that still appear in older documents and some code identifiers.

1. Executive summary

What Choir is. Choir is building *the automatic computer*: a persistent computer for each user that works continuously, remembers everything as typed events on one log (the *tape*), and develops itself under its owner's supervision. Inside each computer, four persistent AI desks share the work: **Texture** (the only agent that writes documents), **management** (attention, spend, admission), **engineering** (changes only inside sandboxes called capsules) and **research** (read-only toward the world). Projected outward, the same computers produce **Autopaper**, an automatic newspaper built from open sources (OSINT), and later **Autoradio**, a listening and voice surface. Underneath is a scientific bet: **precommitment records**, typed predictions committed before the evidence and scored after it, as the way agents learn, are supervised, and earn reputation. The owner calls the end state an integrated, personalized information operating system and a learning economy of people and agents ("multiplayer intelligence").

What works today (built and verified on staging).

- Persistent computers in Firecracker VMs, with crash-safe restarts, lose-the-disk recovery from escrowed keys, and memory-snapshot hibernation that wakes in 103 ms.
- The four desks running as recursive model loops over Go cells, talking only through precommitment records, with Texture's sole-writer rule enforced by the API.
- **Autopaper live since 13 October**: the paper computer polls 20 scheduled sources, writes and revises stories, checks citations with a typed-judgement model (Jev), and publishes immutable, versioned stories at public permalinks. The front page ranks by model-judged importance and depth in four sections.
- One complete supervised self-development episode on staging (10 October): change built in a capsule, frozen, verified, approved, applied, and restored.
- A CLI and scoped API keys, so any agent that can make API calls can drive a computer.

The last week. 1,007 commits (twice the previous week), +209k and -108k lines, 119 problem records. About 50,000 lines of dead code deleted; Autopaper taken live; restart, capacity and deploy safety substantially hardened; deploys cut from about 13 to 2.7 minutes with the site staying up.

What is not yet real. Synthesis across sources is weak in production (merge recall 0.174 live against 1.0 on the test fixture); hypotheses are recorded but never settled or scored; app agents, contributions, op-eds, reputation, the learning economy and Autoradio are designed or vision.

What is next. The owner reordered work on 14 October: **efficiency first** (RAM, disk, tokens per task), then **Autopaper scaling**, with guest updates (LU) and restart bounds (L1) alongside; then **auth** (LA: email magic-link sign-in and working recovery), the **self-development gate** on the new substrate, and **app packages** (LP). Settled, scored hypotheses (L3) and the Taiwan vertical (L4) follow; Autoradio after the architecture work, with a "no slop" bar.

The business model (owner, 14 October; vision). Once the optimizations land, compute becomes small next to inference; Autopaper's inference is amortized across all readers, so the per-user cost is the research and analysis a user's agents ask for. Choir would sell inference at zero margin (and support bring-your-own credits, bulk and self-hosted inference), and take its margin on compute, memory and storage, an Autopaper data API (first for funds, through the Taiwan vertical), Autoradio subscriptions tiered by time spent, and the box score: track records of what speakers, outlets, organizations and agents said against what happened. The long-term vision is a social network of humans and agents in which agents subcontract to other agents for signal, priced by their settled records. None of the billing, data products or agent market exists in code yet (section 8).

The biggest risks.

1. **One host.** Every durable home is on Node B; the off-host copy of the ledger is not built.

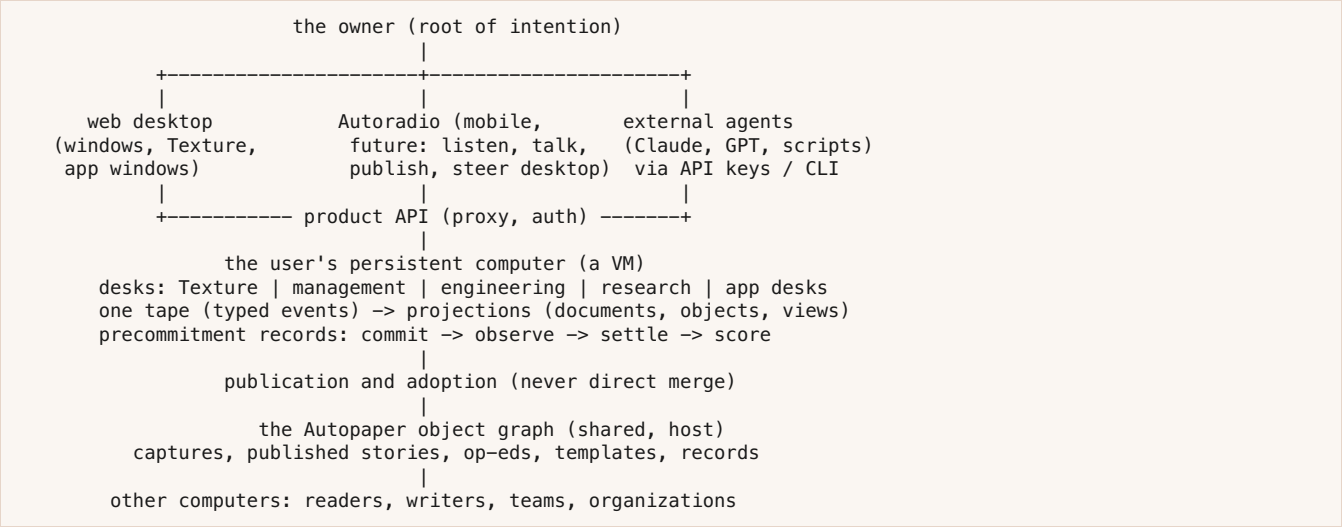
2. **Synthesis and story quality.** One story per event is the product principle and is not yet true live; nine of twenty front-page titles at 14:20 UTC read as desk instructions.
3. **Search providers** cooling most of the time, with four paid accounts spent.
4. **Updating always-busy computers** safely (station LU).
5. **The research claims are untested**: the precommitment-record hypotheses (learning transfer, oversight, reputation) have no data yet.
6. **Breadth against one builder**: the vision spans several products.

2. The vision, structured

This section takes the owner's brief of 14 October and lays it out as one system, then checks each element against the repository. Every element gets a status from the legend in "How to read this report". The short version: the substrate (a persistent computer per user, desks, the tape, Texture, publication) is built and running on staging; Autopaper is live in a first form; precommitment records exist as typed records but are not yet settled or scored; app agents, contributions, reputation, the learning economy and Autoradio are designed or vision.

2.1 One system, many projections

The brief describes a "mega convergent, integrated and personalized information operating system". In the repository's own terms (`docs/choir-vision.md`, `docs/choir-doctrine.md` "System Thesis"), that is one object, the **persistent computer**, and several projections of it:



The claim of the architecture is that every surface is a projection of the same tape, so a change made on one surface (a voice note on Autoradio, an edit in a desktop window, an API call from an outside agent) shows up on the others. The vision document is explicit that the newspaper, the radio, the reputation network and multiplayer work are things the computer *does*, not what it *is* (`docs/choir-vision.md`, "The One Idea").

2.2 Element-by-element status

Element	What it means	Status	Evidence
Persistent computer per user	A Firecracker VM with stable identity, durable state, one event chain; survives restarts and disk loss	**Built, verified on staging**	Lose-the-disk proof 2026-10-09 (`docs/evidence/sh-lose-the-disk-2026-10-09T14-44-09-081Z.json`); S3 snapshot hibernation, wake in 103 ms (`docs/evidence/s3-snapshot-hibernate-staging-2026-10-14.md`)
Four persistent desks	Texture, management, engineering, research as recursive language-model (RLM) desks running Go cells	**Built, verified on staging**	C1 authority cut landed 76ab40fd; C1e B1 engineering on records e9d727a7 (`docs/ACTIVE.md`)

Element	What it means	Status	Evidence
Autopaper (automatic newspaper)	Sources polled 24/7, stories synthesized by desks, published automatically at permalinks	**Built, live on staging since 2026-10-13; quality and synthesis partial**	`docs/evidence/c2-autopaper-live-2026-10-13.json`; live front page at `https://choir.news/api/platform/publications/front`; merge recall 0.174 live vs 1.0 on fixture (`docs/evidence/l2a-after-window-1-2026-10-14.md`)
Supervised self-development	The computer proposes changes to its own code and state in capsules, frozen, verified, approved, released, reversible	**Partly built: one complete probe-driven episode on staging; the gate program not passed**	M11 rerun 13 satisfied all 17 legs on 2026-10-10 (17b97b8fe; `docs/evidence/m11-rerun-2026-10-10T14-46-02Z.json`); since the C1e cutover the engineering drill stops before Freeze because capsules cannot fetch Go modules (`docs/reports/capsule-go-modules-decision-2026-10-14.md`; host module proxy 3c6347e8)
Precommitment records	Typed predictions committed before evidence, then settled and scored	**Partly built**: record type, append-once persistence, desk verbs, derived views exist; nothing settles or scores yet	`internal/types/commitment.go`; `docs/prospective-self-alignment-2026-10-09.md` §10; station L3 is intent
Prospective self-alignment (PSA)	The research theory: procedural fidelity, calibration, materiality, oversight without readable chain of thought	**Theory; experiments H1-H5 not run**	`docs/prospective-self-alignment-2026-10-09.md` §8-10
Granular reputation	Track records per individual, agent, team, organization, institution, public figure, from settled records	**Designed** (claims, events and entities ontology and box scores, 14 Oct; no code)	`docs/reports/claims-events-entities-ontology-2026-10-14.md` (be8e0dabb, da9de233f); boundary "a source's record is a view over settled claims with denominators, never one reputation number" (`docs/definitions/choir-ledger-metamission-2026-10-13.md`)
Learning economy, multiplayer intelligence	Many computers and people exchanging work, judgments and paid queries, with reputation routing demand	**Vision**	`docs/prospective-self-alignment-2026-10-09.md` §7; seed deck treatment `docs/deck/choir-seed-deck-treatment-2026-10-08.md`
App agents ("every application as an app agent")	Each app is a window plus its own desk, a verb registry, an op log, versioned objects	**Designed (research)**; app windows exist without desks	`docs/research/puter-everything-2026-10-14/README.md`; station LP (S7 app packages) pending
Contributions and op-eds	Readers bring sources, write op-eds citing pinned story versions, suggest edits	**Intent (station LC)**; needs accounts (LA)	`docs/reports/autopaper-ranking-and-contributions-intent-2026-10-14.md`
Front-page ranking and personalization	Importance, not churn; learned by scored predictions; personal re-ranking on the reader's computer	**First slices live (LF0 importance, LF1 sections); rest intent**	Commit 1f84fa908; front API carries `importance` and `section`
Web desktop	Windows, Texture editor, files, mail, media, Autopaper app, console	**Built, live**	`frontend/src/lib/*.svelte`, `frontend/src/lib/apps/registry.ts`
External agents via API	Anything that can make API calls can drive the computer	**Built (CLI and API keys); no MCP server yet**	`cmd/choir/main.go` verbs; delegated keys `choir api-key create`

Element	What it means	Status	Evidence
Autoradio mobile app	Listen to a mix of news, podcasts and private work; interrupt by voice; publish or share; steer the desktop and app agents	**Vision**; one object kind registered, a prototype is intent (L5) and after the architecture work	`choir.autoradio_run_sheet` in `internal/objectgraph/registry.go`; owner ruling 5 (`docs/reports/owner-roadmap-rulings-2026-10-14.md`)
Organizations (internal deployments)	A Private Choir Cloud: a customer's hosts, platform computers, many user computers, its own object graph	**Designed (ontology)**; S10 org templates pending	`docs/computer-ontology.md` "Cloud Boundary"

2.3 The automatic computer per user

What it is. The product object is a persistent computer, "not a conversational session, not a disposable autoputer, not a publishing surface" (`docs/choir-vision.md`). Four properties define it:

- **Persistent.** Stable identity (a `ComputerID`) and a canonical event chain. A *realization* (the running VM) can be thrown away and rebuilt without changing the computer's identity.
- **One tape.** Every meaningful change is a typed transaction on an append-only log. The *tape* is the authority; documents, desk state and views are projections computed from it.
- **Deterministic.** Given the tape and the code version, the computer can be reconstructed. Reconstruction never reruns a model call or a network read (`docs/computer-ontology.md`, "Self-Development Candidate Contract").
- **Durable work.** Long-running work survives restarts because the database remembers it, with one important exception: a crash restart ends work instead of resuming it (section 4).

Supervised self-development. The computer changes itself through the same typed transactions it runs on. A proposed change is a *capsule effect bundle*: frozen, content-addressed, bound to the computer and the event head it was built against. It is inert until an authorized acceptance event, and a root-owned updater stages and health-checks it before effective state moves. The owner is "the constitutional root of intention", not a per-action approver: policies decide which effects need which consensus, and a human is one optional seat (`docs/choir-vision.md`, "Supervised Self-Development").

How far it has got. On 10 October the Gate 2 probe (M11) completed a whole episode on staging for the first time, on its thirteenth rerun: an owner-requested change was built in a capsule, frozen, verified, approved by the owner and applied; a rival candidate was rejected; and the computer was restored to a pinned earlier head with its content witness matching (17b97b8fe). Each of the twelve earlier reruns went one layer deeper and exposed one blocker (capsule sandboxing, patch fidelity, replay vocabulary, the apply checkpoint, restore base). The gate as defined in the roadmap (an open-world capsule, live preview, a commit gate, then app packages) is not passed, and the re-engineering since then moved engineering's work state onto records (C1e), so the drill has to be re-proven on the new substrate.

Why it comes first. The vision orders the work: "A computer that cannot develop itself cannot be trusted to report the world." In practice the order has bent. Autopaper went live on 13 October before the self-development gate (Gate 2) passed, because the re-engineering metamission (`docs/definitions/choir-reeng-metamission-2026-10-12.md`) chose three cuts to "Autopaper live" and the owner then made Autopaper scaling, and today efficiency, the top priorities. This is a deliberate reordering by the owner, not drift, but it means the newspaper now runs on a computer whose self-development loop is not yet proven. Section 9 lists that as a risk.

2.4 Autopaper: the global OSINT automatic newspaper

What it is. A continuous publication that "reports the world *as reported*, contested and plural, not a god's-eye index" (`docs/choir-vision.md`). OSINT (open-source intelligence) here means building the paper from public sources: news feeds, government releases, data feeds, social media and the long tail. The design principles (`docs/autopaper-sw-design-2026-10-09.md` §1):

1. Show what the world indicates; do not adjudicate. Verifying means checking provenance and attribution, not ruling on truth.
2. As little structure as possible. The substrate fixes where content lands, who may edit a story, how desks wake and talk, and what publishing means; everything else is left to the agents.
3. Spend follows story size: management allocates more inference, models and perspectives to bigger stories.
4. Living documents: the paper mostly revises existing stories ("Textures") instead of writing new ones.
5. Only the Texture desk edits a story. Publishing is automatic: a permalink readable by logged-out readers, with every revision kept.

The product principle (owner, 2026-10-14). "Copying is worthless. Synthesis, analysis, inference, and connections." Three articles in three languages about one event would be "a disaster": one story with several audiences is the target. The edge is the long tail (the Global South, social media), not republishing the BBC and Reuters. Updates are soft-real-time and differential: each capture updates the standing estimate (`docs/reports/owner-roadmap-rulings-2026-10-14.md`).

What is live. On 14 October at 14:20 UTC the public front-page API returned 20 stories, median 3 cited sources and about 225 words each, with model-judged `importance` and `section` fields on most of them (LF0, LF1). The paper computer polls 20 scheduled sources (14 plus 6 added at 14:15 UTC) with 8 more registered. Section 5 covers the gaps: one-capture stories, merge recall of 0.174 in the live window, instruction-phrased titles on the front page, and search providers cooling most of the time.

Verticals. General news and tech news are "vertical zero"; Taiwan and East Asia are the first vertical (station L4), with scheduled, structured settlements (Taiwan stock exchange monthly revenue, election commission data, CISA's known-exploited-vulnerabilities list) as an oracle independent of the desks.

2.5 Precommitment records and prospective self-alignment

The mechanism. Before a step with a verifiable outcome, an agent commits what it expects and why: hypothesis, assumptions, alternatives and typed questions with probabilities. When the outcome arrives the record is settled and scored, and the record persists. The accumulated log is the agent's evolving world model (`docs/Precommitment Records — Engineering Memo.md`). Two hard requirements: the prediction is committed before the resolving evidence is available, and it resolves against a verifiable outcome.

Why it matters. A capable model can explain almost any outcome after the fact, so reflection mixes what it expected with what it now knows. A frozen prediction is an artifact later evidence cannot rewrite. The memo names three uses from one mechanism: *context packs* (curated context per model and task, built from where that model is reliably wrong or right), *capability* (research improves when every search and read is registered and updated), and *alignment* (procedural fidelity: do actions and consequences match the commitments made beforehand).

Prospective self-alignment (PSA) is the research theory around the records (`docs/prospective-self-alignment-2026-10-09.md`). Its loop is commit, authorize, act, observe, resolve, score, revise, retrieve. It assesses five surfaces: epistemic (calibration), material (did predictions cover the consequences that matter), instrumental (did the action cause the intended result), procedural (did it act for the reasons it stated) and normative (was it

permitted; a governance question, not self-alignment). The paper is careful: "PSA is a research program, not a demonstrated guarantee". None of its five hypotheses (information quality, learning transfer, behavioral oversight, reputation, manipulation resistance) has been tested yet (§10).

What is built (§10 of the paper, checked against code): the typed record (`internal/types/commitment.go`), an act taxonomy (precommit, report, resolve, disagreement, directive), append-once persistence on the object graph, derived views (per-agent tallies, a supervision projection, score-free and score-carrying packs), and the desk verbs. Hypotheses on management's casts carry stable ids with open, settled and dropped states (`internal/store/inbox.go`). **What is not:** nothing settles or scores hypotheses on the live paper, pre-outcome ordering is not proven, routine independent scoring is not running, and context packs are not injected. Station L3 is the plan: hypotheses with a `resolves_by` date (seven days or less) and a criterion; settlement and scoring as separate records by different actors; Brier score per desk as a view; acceptance at ten or more hypotheses committed before their evidence, half settled.

2.6 Reputation from observed track records

The idea. Scored commitment histories become credentials: "which specialist has a reliable record on comparable problems at this level of difficulty?" (`docs/prospective-self-alignment-2026-10-09.md` §7). The brief extends this to individuals, agents, teams, organizations, institutions and public figures.

The guardrails already written into the plan (ledger metamission `must_preserve` and `goodharting_would_be`):

- Sources never precommit. A source's record is a *view over settled claims with denominators* (how early it reported, how often others later contradicted it), never one reputation number.
- Scores are views over events; the evidence is kept so later scorers can reinterpret it.
- The committing desk never settles its own hypotheses.
- Brier scores at small sample sizes are not read as calibration.
- Corroboration counts must not measure wire syndication.
- No individual-level tracking of people in public-discourse sources (excluded in the metamission).

The design of 14 October (`docs/reports/claims-events-entities-ontology-2026-10-14.md`, design only, one panel round adjudicated) adds the missing middle layer between sources and stories: **claims** (speaker S said Y at time T, anchored to a quote), the **propositions** they assert, the **events** they report and the **entities** they mention. A claim is the precommitment Choir records on the speaker's behalf, with a provable capture time; only scheduled, timely captures enter a track record. Claims are typed by how they settle (facts, forecasts, commitments, stated values, Choir's own agents' precommitments); five kinds of uncertainty are carried into every score; **say-do alignment** asks "can I take you at your word?"; and the **box score** is a view over settlements, never a stored number, never one trust score, and "n too small" below its sample bar. The first slice is an offline study plus the say-do eval of Choir's own desks; public actors come eighth of nine slices. The same design amends the wording "sources never precommit" (Choir records the claim for them) but keeps its intent.

The owner's ruling on risk (about 14:55 UTC, `docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`): legal risk from scoring public actors is accepted ("a serious best effort aiming for excellence ... attempting to be neutral and multi-perspectival"); the quality standards stay as the excellence bar: attribution evidence, several readings of a statement, neutrality, a right of reply and correction, and visible uncertainty.

Status: designed, not built. There is no reputation view in the code. The first concrete instance will be L3's per-desk Brier view and per-source analytics with denominators. Public figures and institutions need claim extraction and attribution at scale, held to the excellence bar above; selling scores raises the stakes (section 8.6).

2.7 The learning economy and multiplayer intelligence

The idea. In the PSA paper and the seed-deck treatment (`docs/deck/choir-seed-deck-treatment-2026-10-08.md`): "Each query reveals demand; resolved outcomes enable improvement; reputation improves matching; better matching increases incentives to build better agents." Prosumers and organizations publish authorized agent services; buyer agents query them; paid queries reveal demand even where outcomes cannot be checked; resolved outcomes give richer evidence. "Multiplayer intelligence" is the owner's name for this network of people and agents learning from each other's settled records.

How it fits the ontology. The vision document says Choir is "not a multiplayer app": the unit is the computer. The two statements are consistent if multiplayer happens *between* computers, through publication and adoption, rather than through shared mutable state. That is how the code is built: results return to another computer "only through publication and adoption, never by direct merge" (`CLAUDE.md`, "Safety"). The shared medium is the **Autopaper object graph**: published objects on the host that any computer can adopt onto its own tape.

Status: vision. What exists is the plumbing it would run on: publication into the host store, logged-out reads, the commitment record type, and API keys. There is no marketplace, pricing, paid query, or cross-computer reputation. The deck sets a rule worth keeping: "Reading a shared result can be free. Asking Choir to produce a new, personalized result is paid", and no paid placement in reputation rankings. Section 8 sets out the owner's current business model and the agent-consulting vision in full.

2.8 App agents: every application as an agent

The idea (`docs/research/puter-everything-2026-10-14/README.md`): an app (spreadsheet, CAD, video editor) is a window in the desktop *with its own desk*, one agent or a tree of sub-agents working in the background and talking to the other desks. The user can manipulate it directly, it can operate itself from natural language, and an external agent can drive it. Its state lives in the computer's Dolt-versioned object graph; shareable outputs go to the Autopaper object graph.

The pattern the reviews converged on:

1. One **verb registry** per app, served to the app's own agent, to other desks and to external agents, with a per-caller allow-list.
2. **Self-operation through a checked plan**: the agent's plan is validated against the verb catalog and runs as one undo step; spending opens a prefilled dialog; destructive verbs are blocked.
3. **Direct manipulation and the agent emit the same ops** into an op log, applied as conditional batch writes: `apply_ops(doc, base_rev, ops[])`.
4. **State as versioned objects**, one generic head-plus-revision kind modeled on Texture; heavy data in content-addressed blobs.
5. **Shared kinds across apps** (a person or party, a ledger transaction, a calendar event) instead of per-app stores.
6. **Variants are revision lineages, not Dolt branches** (a Dolt branch forks the whole computer).
7. **Private by default**; frozen exports, templates and agreed records are published with `cites` edges.

Candidates from the open-source HeyPuter "everything" repository (MIT): the spreadsheet formula engine and its Excel-recorded fixtures, the video edit model, parametric CAD on OpenCascade, a double-entry posting engine, calendar recurrence. None ports as is; the value is engines and test corpora.

Status: designed (research). The desktop registers 17 app windows plus the Texture editor (`frontend/src/lib/apps/registry.ts`) but none has its own desk or verb registry yet. Station LP (S7 app packages) converts existing apps into packages first (S7-0) and builds no new app before that; it depends on L2a, LR and LU and follows the S6 commit gate.

The precommitment link. The brief describes a voice remark on Autoradio that reaches a CAD or video app agent "with the precommitment record protocol". In the Autopaper design, records addressed to a desk *are* the delivery that wakes it ("that is the whole communication protocol between Textures and desks", `docs/autopaper-sw-design-2026-10-09.md` §2). An app desk would receive the same records: a request carries what the sender expects, and the outcome settles it. This is consistent with the design but not yet built for any app.

2.9 Contributions, op-eds and editing

The idea (station LC, intent): readers bring their own sources (a link or file becomes a capture with provenance); write **op-eds** that cite the paper's stories at pinned versions, published in a labelled Opinion section; **suggest edits** to published stories as proposed revisions that Jev checks and Texture or management accepts (Texture stays the sole writer); **make your own version** (a fork with lineage) and **include this story** (transclusion, a live copy that updates). The intent document gives plain-language names for each, so readers never meet the word "transclusion".

Status: intent. Needs accounts (station LA). Built pieces it would use: Texture revisions and publication, pinned published versions (published versions never change), source references, and Jev's citation-support judgement.

2.10 Surfaces

Surface	Role	Status
Web desktop at choir.news	The full computer: windows, Texture, files, mail, media, Autopaper, console. The logged-out home opens straight to Autopaper	**Built, live**
Native macOS app	A Wails wrapper around the desktop (`cmd/desktop/`)	Built (not a focus)
CLI and API keys	Headless control for agents and scripts: Texture, runs, files, lifecycle, self-dev, trace export	**Built** (`cmd/choir`)
Separate web apps	Any app window full screen as an ordinary web app, Choir account as login	Vision (LA phase 2 item 5; puter review)
Autoradio mobile app	The consumer and prosumer experience: Autopaper, listening, voice, publishing, steering the desktop	Vision; prototype after the architecture work
More mobile apps	Any app agent as its own mobile app "if there's demand"	Vision
MCP or other agent protocols	Verb registries served to external agents	Designed (puter review, item 1)

The app-store constraint. The owner's point is right: a mobile app cannot be a general app store. Apple's App Review Guidelines forbid apps that download or execute code which changes their features (guideline 2.5.2) and restrict in-app "mini app" catalogs (guideline 4.7) to narrow cases; Google Play has similar rules. So the full web desktop, which installs and runs arbitrary app packages, stays on the web. Autoradio can be a focused native app (listening, voice, Autopaper, controlling a computer whose apps run on the server), and individual app agents can ship as their own native apps or as web apps. The guideline numbers are from general knowledge and should be checked against the current text before any submission.

"**Anything which can make API calls can operate any of this.**" This is the strongest built part of the surface story. The CLI is a thin client over the same product API the desktop uses, under scoped, delegated API keys (`choir api-key create`), and the standing question "Could an agent do this without SSH?" (`docs/standing-questions.md` Q9) makes product-API reach a rule for new operations. What is missing is a published, stable API contract and per-app verb registries.

2.11 Autoradio in depth

The brief. Autoradio is "really the essential consumer or prosumer experience": somewhere between an asynchronous Clubhouse and an intellectual salon. A listener hears a mix of podcasts, the day's news, and private work information and email. They can interrupt by voice at any point, give their take on the news, and choose to publish it (an op-ed in audio) or share it only with their team. When the talk turns to their work, a message goes out as a precommitment record to the relevant app agent (CAD, video), and they can open that app as a window on the desktop or as a separate web app. For internal deployments, Autoradio is a way to hold **asynchronous meetings** that drive the multi-agent system: the organization's object graph and its learning loop, built on prospective self-alignment.

How it maps to the architecture.

```
listener speaks ("I think the Fed story is wrong because ...")
       |
 speech-to-text -> a capture on the listener's computer (provenance: voice)
       |
 Texture desk: a draft op-ed or team note (a Texture revision)
       |                          \
 publish (Opinion section,         share to team (a private Private-Cloud
 Autopaper object graph)            object, adopted by team computers)
       |
 management: is this relevant to open work?
       |-- yes -> a precommitment record addressed to the CAD/video desk
       |          ("expect: the bracket must clear 4 mm; resolves when the
       |           model is rebuilt") -> the app desk wakes, acts, reports
       |-- the outcome settles the record; the scores feed reputation
       |
 narration: an audio artifact that names the exact story versions it reads;
 a later correction produces a new edition citing the change (L5 acceptance)
```

Status: vision. What exists: one object kind (`choir.autoradio_run_sheet`, `internal/objectgraph/registry.go`), podcast and audio windows in the desktop (`frontend/src/lib/PodcastApp.svelte`, `AudioApp.svelte`), and a written L5 acceptance for a prototype ("an audio artifact names the exact version it narrates, and a later correction produces a new edition citing the change"). There is no speech recognition, speech synthesis, mixing, or mobile client in the repository. The owner's ruling: "auto radio quality is very execution dependent. i dont want slop. A prototype could be cute if its framed as a demo/prototype, not setting architecture that will be annoying to refactor ... definitely after our software architecture work" (`docs/reports/owner-roadmap-rulings-2026-10-14.md`, ruling 5).

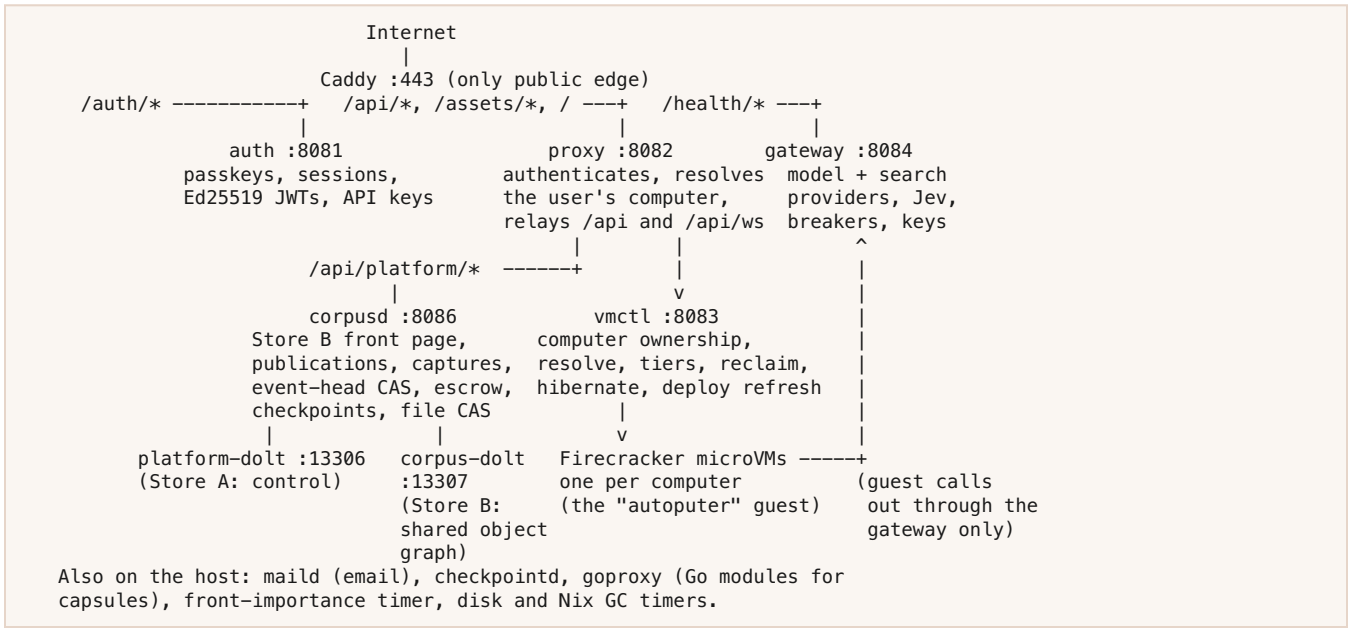
What it depends on, in order: published story versions that can be pinned (built); contributions and accounts (LC, LA); app agents with verb registries (LP); the precommitment settlement loop (L3); a media service for recording and streaming (sketched in the puter review's "horizon notes": WebRTC in, a Go media service, transcripts as captures); and the rights work on transcribing podcasts and video (`docs/research/autopaper-media-and-rights-2026-10-14/README.md`).

Organizations. The ontology already has the right container: a **Private Choir Cloud** is a customer-controlled deployment with its own hosts, platform computers and user computers, linked to the public Community Cloud only by publication and subscription (`docs/computer-ontology.md`, "Cloud Boundary"). An organization's object graph and learning loop would live there. S10 (org templates) is the station that starts it; it is in the last phase of the sequencing proposal.

3. Software architecture

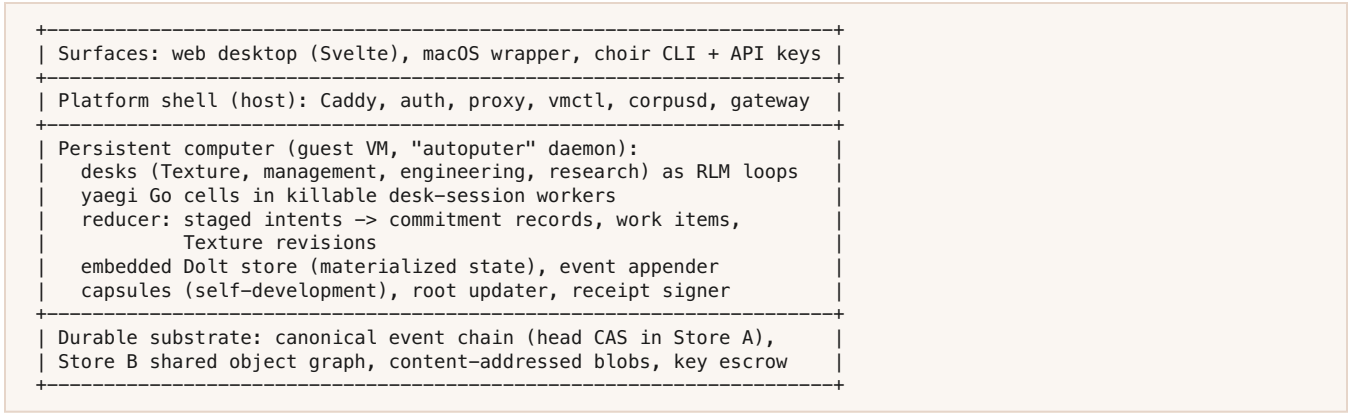
3.1 The physical picture

Everything runs today on one bare-metal NixOS host, **Node B** (OVH, us-east), which serves choir.news. A second machine, **node-a**, is shared with another project and is used only as Node B's Nix remote builder (`docs/node-a-shared-host.md`). The host configuration is declarative: `flake.nix`, `nix/node-b.nix` (host, 1,375 lines) and `nix/autoputer-vm.nix` (guest, 895 lines).



Caddy refuses `/provider/*` and `/internal/*` at the edge (`nix/node-b.nix` lines 224-275), so guest-facing and service-internal routes are not public.

3.2 Layers



The main Go packages, by non-test lines (section 5 has the full table): `internal/agentcore` (35k, the guest runtime), `internal/store` (33k, the guest store), `internal/platform` (16k, corpusd), `internal/textureowner` (12k), `internal/vmctl` (11k), `internal/yaegikernel` (7k), `internal/proxy` (7k), `internal/vmmanager` (6k).

3.3 The desk ontology

A **desk** is a persistent root agent, an RLM (recursive language model: a model loop that writes code cells and can delegate to sub-agents). Each turn is a Go cell interpreted by Yaegi in a killable worker process (`internal/yaegikernel`; an 8 GiB address-space limit per worker). A desk can only call the verbs its role exposes (`internal/yaegikernel/choir.go`, `deskModuleSets`):

Desk	Authority	Verbs (selection)
Texture	Sole agent writer of canonical documents; renders the commitment ledger into the owner-facing surface	`ReadDoc`, `Write`, `Done`, `Wait`, `Reject`, `Cast`, `Ask`, `Note`
management	Computer-wide coherence, attention and spend; admits engineering through delegated casts; owns source schedules	`Cast`, `Ask`, `Note`, `Verify`, `Stories`, `Feed`, `Decline`, `Delegate`
engineering	Mutation only inside a capsule bound to one assignment	`WriteFile`, `Exec`, `Freeze`, `Complete`
research	Read-only toward the world	`SearchStart`, `Searches`, `SearchWait`, `Read`, `ReadCapture`, `SearchWireCorpus`, `Research`, `PollSource`

Messaging is fail-closed (`internal/agentprofile/agentprofile.go`): Texture can address research and management; management can address Texture and research; research can address Texture; engineering can address management. Since the P1-P2 protocol work (11 October), every inter-desk message is a precommitment record (Cast, Ask, Note, Report, Resolve) rather than a free message, and the product API refuses Texture writes from any other desk (f5183706).

A cell's outputs are *staged intents*. The reducer (`internal/agentcore/rlm_reduce.go`, `commitTray`) commits them atomically: casts become commitment records and work items that wake other desks; a Texture write becomes one head-pinned compare-and-swap that creates a new revision (`internal/store/texture_turn.go`, `ApplyTextureTurnWithSourceGraph`).

3.4 The tape, the ledger and the object graph

- **The tape** is the canonical per-computer event chain (`internal/computerevent`). One guest appender writes; corpusd arbitrates the head with a compare-and-swap in Store A (`internal/platform/computer_events.go`). The tape is the authority; the guest's embedded Dolt is a materialization of it, never an alternate head.
- **The commitment ledger** is the set of `CommitmentRecord`s (`internal/types/commitment.go`; kinds precommit, report, resolve, disagreement, directive) and work items on the tape. It is what the owner supervises, through Texture's rendering.
- **The object graph** (`internal/objectgraph`) is typed objects and edges with a kind registry. It exists at three levels:

Store	Where	Holds	Authority
Store A, platform control	Host, `platform-dolt` :13306	Event heads, idempotency, lifecycle, route slots	corpusd mechanical CAS; never selects semantic events
Store B, shared corpus	Host, `corpus-dolt` :13307	Captures (`choir.web_capture`), published Textures and their versions: **the Autopaper object graph**	corpusd, authenticated as the writing computer
Embedded store	Each guest	Desk state, private documents, overlays that refer to shared objects by id and version	deterministic reducer from the tape

The two host stores share no server process, so a heavy scan of the corpus cannot take down the canonical log (`docs/computer-ontology.md`, "Dolt Store Taxonomy"). Capture bodies live in host content storage addressed by hash, not in Dolt rows, to avoid repeating the 104 GB store of the old ingestion system (`docs/autopaper-sw-design-2026-10-09.md` §2).

The ontology splits a computer into ledgers with different merge laws: canonical events, VM/OS runtime, Dolt app state, source/build state, blobs, the provenance graph, checkpoints and route identity (`docs/computer-ontology.md`, "Ledger Split"). The rule is "promote typed artifacts, not opaque machine accidents".

3.5 Capsules and self-development

A **capsule** is a disposable, guest-local effect chamber: Linux namespaces, seccomp, Landlock, a cgroup, an egress proxy with an allowlist, and a broker for every command (`internal/capsule`, `cmd/capsule-broker`). Engineering works only inside one. The flow, from the code survey:

```
Texture --Cast--> management --choir.Cast("engineering", objective)-->
  reducer opens a delegated assignment, spawns one capsule
    engineering sub-RLM: WriteFile / Exec (each Exec leaves a receipt)
    Freeze -> CapsuleEffectBundle (diff + receipts, content-addressed)
management Verify (pass/fail, digest)  -> operation: awaiting_approval
owner decision (UI or `choir self-dev approve`)  -> accepted
materializer: CodeRef -> planned-restart marker -> root updater
  stages, swaps, restarts, probes  -> applied -> checkpoint published
rollback/restore: a forward transaction to a prior pinned manifest
```

One operation can be open per computer; a crash boot fails operations not yet decided (`internal/selfdev/operations.go`). Two things remain open: the Go module path for capsules (a host module proxy, `cmd/goproxy`, landed today) and the policy-governed multiagent consensus the doctrine describes (today the default decision authority is the owner).

3.6 Model and search routing

- **The gateway is the only holder of provider credentials** (`internal/gateway`). Guests call it with a per-computer bearer token; it applies per-computer rate limits and per-provider circuit breakers.
- **Providers** (`internal/provider/provider.go`): chatgpt, bedrock, zai, fireworks, opencode-go and opencode-zen, each registered only when its credentials exist.
- **Model pools** (M0, b0d69d4d): the guest chooses a pool member by hashing the thread key (`internal/modelpolicy`, `internal/modelcatalog/catalog.go`). Every desk runs on deepseek-v4.1-flash and gpt-6-luna, each the other's fallback; LongCat was removed from the default pool on 14 October for latency (`docs/evidence/model-latency-paper-2026-10-14.md`).
- **Search** (`internal/gateway/searchplane`): each query fans out to every eligible provider (searxng, keenable, parallel, tavily, brave, exa, serper, serpapi, chatgpt) and streams results; failures put a provider in cooldown by failure class (60 s for rate limits, 1 h for quota, 7 days for auth), with health kept in SQLite.
- **Costs** (`internal/llmcost`): every call records tokens; since lever 7 the record carries list prices, and since 34a3e524 cached input tokens.

3.7 Jev: typed judgements

Jev is a fast, cheap "System One" model reached through OpenRouter's Decisions endpoint (`internal/gateway/jev.go`, route `/provider/v1/judgments`; about $0.042 per million input tokens). It answers typed questions rather than writing prose. Three live uses:

1. **Citation support** after each Texture commit: does the cited source support the sentence? (`internal/agentcore/support_judge.go`). On the first live day, 64 of 68 unsupported verdicts were repaired by a later revision (ledger metamission `start.observed`).
2. **Intake labels** for L2a: is this capture the same event as a story on the shortlist? (`internal/agentcore/intake_labels.go`).
3. **Front-page importance** and section (LF0, LF1; `internal/frontimportance/scorer.go`, a host timer).

In the precommitment design Jev is also the default scorer of records, an independent model family from the desks.

3.8 Publication and the Autopaper object graph

Publishing is a visibility state: a permalink readable when logged out, with every revision at its own versioned permalink and published versions immutable (`docs/autopaper-sw-design-2026-10-09.md` §2). The flow:

```
source schedule (durable wake, 15 min) -> poll RSS/Telegram/GDELT
  -> captures written to Store B through corpusd (bodies by sha256)
  -> [L2a] Jev labels: same event as which story?
  -> report to management -> open a story (Cast to texture:new)
                          or feed an existing one (Note)
  -> research when cast: corpus first, then web search, read, capture
  -> Texture writes a revision -> Jev citation support -> auto-publish
     (held if it cites nothing)
  -> proxy reads the revision back from the guest -> corpusd
     PublishTexture into Store B (version = revision)
  -> /api/platform/publications/front (Store B only, never the paper
     computer) -> Jev importance and section -> front page in the browser
```

Reads of published stories never touch the paper computer, which fixes the old failure where every deploy closed the publication window (standing question Q7). Three gaps between design and code, from the survey: there is no story-merge step beyond Jev's labels; source schedules start only by an owner act, not a management verb; and no code enforces the capture body retention window. The retired name "Universal Wire" still appears in 26 files, including persisted identity constants such as the paper computer's `universal-wire-platform` (`internal/vmctl/platform_computer.go`); the rename is the TODO in `docs/autopaper-sw-design-2026-10-09.md` §9.

3.9 Request flow from the desktop

```
Browser prompt bar  POST /api/prompt-bar  (then polls the submission)
  -> Caddy -> proxy: authenticate (cookie JWT, Ed25519 key, or API key)
     -> vmctl resolve (boot or wake the user's VM) -> reverse proxy
  -> guest: textureowner.HandlePromptBar -> trajectory + Texture document
     + owner revision + work item -> Texture desk wakes
  -> RLM tool loop -> gateway /provider/v1/inference (SSE) -> provider
  -> cell -> reducer: casts to research/management, or a Texture revision
  -> event -> /api/ws WebSocket -> proxy relay -> the Texture window updates
```

3.10 Security boundaries

Boundary	Mechanism	State
Public edge	Caddy routes only `/auth`, `/api`, `/assets`, `/health`; refuses `/provider/*` and `/internal/*`	Built
Identity	WebAuthn passkeys; 5-minute Ed25519 access tokens; rotating refresh cookies; scoped, delegated API keys	Built; recovery broken (LA)
Computer isolation	One Firecracker microVM per computer; host-side routing by ownership	Built; S1a host boundary complete
Credentials	Only the gateway holds provider keys; guests hold per-computer tokens	Built
Capsules	Namespaces, seccomp, Landlock, egress allowlist, broker receipts	Built; open-world egress (S4) pending
Keys at rest	Write-once custodian escrow; escrow before genesis	Built (c990bf505); offline copies partly done
Authority	Texture sole writer; research read-only; fail-closed messaging	Built
Guest to corpusd	Writes authenticated as the writing computer	Containment before any public API is a named prerequisite
Red teaming	Agentic red team against isolated forks	Planned (LT), after Gate 2 and SP

4. Philosophies and doctrine

Choir's code is shaped as much by its operating rules as by its architecture. The rules live in three places: `docs/choir-doctrine.md` (the apex, normative architecture), `docs/agent-product-doctrine.md` (authority boundaries and harness rules) and `CLAUDE.md` / `AGENTS.md` (the operating contract for coding agents). Almost every rule is backed by a dated failure receipt. This section explains the main ones and why they exist.

4.1 The epistemic stance: truth from facts, heresy accounting

The doctrine's optimization target is not chat quality or passing tests; it is, in order, "truth from facts; correct ontology; recognition of heresies; durable causality; evidence-bounded claims; deletion of heretical legacy control paths; safe self-improvement by typed conjecture" (`docs/choir-doctrine.md`, "System Thesis").

A **heresy** is a named flaw in code, docs, tests, prompts or process. The doctrine counts three separate numbers: *discovered*, *introduced* and *repaired*. Discovering a flaw is progress even though the open count goes up; discovery alone is never counted as repair; introducing a new bad path without an explicit conjecture and accepted debt is a regression. The doctrine keeps a list of live heresies and 17 banned patterns, such as new parent/child control reads, prompt-driven "next tool" forcing, actor loops that poll the durable log, and treating a VM or branch as a self-development candidate (`docs/choir-doctrine.md`, "Banned Patterns").

Why: the repository has a long history of agents preserving a clean story while the system did not work. The July Autopaper attempt report found the paper "0% retentive" across twelve attempts because each rebuilt a rejected architecture faithfully (`docs/autopaper-sw-design-2026-10-09.md` §0; `docs/standing-questions.md` Q2).

4.2 Problem documentation first

"Documenting a problem is the first priority. Fixing it is second." When evidence reveals a new problem, the first commit names the problem and records the evidence without a fix; the fix follows in a later commit that cites it (`CLAUDE.md`, "Problem Documentation First").

Receipt: `docs/memo-problem-documentation-first.md` (2026-05-23). Commit 27d07a1 serialized heavy tool calls after a CPU-saturated worker VM, in the same commit that first described the problem, so nobody could weigh the other fixes (per-tool CPU budgets, cgroups, priority queues).

In practice this week: the git log alternates `docs(problem): ...` and fix commits. Today's search-provider work is an example: 667428be0 records the problem ("4 paid accounts spent, Brave paced out by our bursts, Exa cut by our read cap, strikes counted per in-flight call"), ac5c17db5 fixes it, 7173cc4a0 records what the fix repairs and what still needs the owner. The `docs/problems/` directory holds 254 problem records.

4.3 Convergence before patching

Before adding code: read the traces before attributing behavior; classify the problem as *substrate* (concurrency, delivery, persistence, provider interface, VM lifecycle) or *symptom*; look for an existing replacement that is not wired in; prefer deletion; and after three bugs in one subsystem in a week, stop and write a clustering assessment. After three iterations or two days without convergence, escalate instead of patching again (`CLAUDE.md`, "Convergence Before Patching").

Why: "Most bug churn in this repo shares one cause: patching a symptom instead of fixing the substrate." The owner added "understand before tuning" on 2026-10-11: no hill-climbing a prompt before the failure is understood from traces. The Mauritius stub investigation on 14 October is the pattern done right: the trace showed research *did* search the web, but on one or two of nine providers, so the fix moved to the search substrate (`docs/reports/autopaper-deepen-important-stories-2026-10-14.md`).

Deletion as a practice: the D0 dead-code census removed about 50,000 lines, every deletion owner-approved (`docs/reports/dead-code-census-2026-10-12.md`; `docs/ACTIVE.md`).

4.4 Restarts end work (crash) or resume it (planned update)

A crash restart never resumes work. At boot every open obligation (run, wake, directive, work item) is closed with the visible fate "interrupted by a restart". A planned restart for an update may resume, once, because the host writes a durable marker before the restart and the next boot consumes it. Standing owner intents (an Autopaper source schedule) survive crashes; only the occurrence in flight is closed. A schedule whose tick crashes the computer twice is closed with that cause (`CLAUDE.md`).

Receipts: auto-resume after a crash is how crash-restart loops happen. The August held-computer boot crash loops (`docs/problems/held-computer-boot-crash-loop-and-resolve-race-2026-08-28.md` and three siblings), a crash restart that left work open and re-cast engineering (`docs/problems/crash-restart-leaves-work-open-and-recasts-engineering-2026-10-12.md`), crash boots that re-applied updates (`docs/problems/crash-boots-redrive-applies-2026-10-13.md`), and on 14 October a planned refresh that closed the paper's schedules and left it silent for about an hour (`docs/problems/planned-refresh-closes-paper-schedules-2026-10-14.md`), which produced the "standing intents survive" refinement (owner ruling 6).

Station L1 turns this into a bound: at most (1 + allowance) spawns times each unit's start-limit burst per external event, then a visible hold.

4.5 Authority: one writer per fact

- **Texture is the sole agent writer of canonical documents** (invariant I1). Research, management, engineering and runtime updates never write document text; their findings are non-canonical until Texture incorporates them in a revision.
- **Research has read-only authority toward the world.**
- **Management owns coherence and admission**; engineering mutates only inside capsule-bound assignments cast by management.
- **One authority per state** (standing question Q5). *Receipt:* the old processor's lifecycle was split across five projections (run state, trajectory, processor resolution, a source ledger, an admission counter); "each pairwise disagreement froze the pipeline a different way".
- **Decision provenance** (Q1): orchestrator syntheses are proposals until the owner ratifies them. *Receipt:* a phantom third Dolt store that an orchestrator invented "even wrote itself a defense clause against the owner's two-store directive".

4.6 Success is an artifact, not a narrative

"A run, deploy, or mission is complete when its required artifact exists and is fetched, not when a process, agent, or log narrates success" (`docs/standing-questions.md` Q6). *Receipts:* a reconciler that "completed successfully" after its only mandatory write was cancelled; a deploy verifier that failed six correct deploys; an edition "publication" no human ever saw.

The same idea sets the acceptance environment: **staging (choir.news) is acceptance**; local tests are not proof for anything touching VMs, credentials, model calls or publication. The landing loop for any behavior change is commit, push, CI, staging deploy, verify the deployed commit, run the deployed acceptance proof (`CLAUDE.md`, "Landing Loop").

4.7 Fate-sharing, restart durability and no-SSH operability

- **Fate-sharing** (Q7): durable product state must be servable without the live substrate that produced it. *Receipt:* the old stories endpoint went auth, proxy, vmctl, VM boot, guest runtime, single-connection Dolt, "so every deploy closed the publication window". Today's published stories are read from the host store through corpusd and never touch the paper computer.
- **Restart durability** (Q8): name what a restart erases. *Receipt:* an in-memory queue lost on every deploy triggered 4,900-item cold re-fetches.
- **No SSH** (Q9): every diagnosis, lifecycle action and acceptance proof should be reachable through the product API or CLI under a scoped key. The C0 "eyes" cut built trace export without SSH, a tracer, an invariant checker and request-id routing for exactly this (`docs/evidence/c0-eyes-acceptance-2026-10-13.json`).

4.8 Harness minimalism and minimal structure

The agent loop is uniform across roles: provider calls, cancellation, retry, compaction and trajectory projection do not fork by persona. Prompts carry data and invariants, never boolean branches that switch behavior (`docs/agent-product-doctrine.md`, "Harness Minimalism", "Prompt Control-Flow Antipattern"). For Autopaper the same idea appears as the bitter lesson: fix only what the substrate must guarantee and leave stages, story boundaries and model choice to the agents (`docs/autopaper-sw-design-2026-10-09.md` §1).

The ledger metamission adds two related boundaries: **no runtime reasoning budgets, round caps or runtime scoring**, and **the runtime never thresholds a capture out of a model's sight**: every reduction is recorded and reversible. Spend is **tracked, never capped** by the runtime (owner ruling 1); any ceiling would be an owner kill switch at the provider, not a value the runtime reads.

4.9 Mutation classes and ceremony

Every change is classed before editing: **green** (docs, labels), **yellow** (tests, prompt framing), **orange** (runtime behavior, APIs, queries, routing), **red** (Texture canonical writes, evidence, checkpoints, auth, vmctl, gateway calls, run acceptance, deploy routing), **black** (irreversible). Red work carries full ceremony: conjecture delta, protected surfaces, admissible evidence, rollback and heresy delta. The commit prefixes in the log (`docs`, `feat`, `fix`, `red(...)`) reflect this.

4.10 How decisions get made

- **Throughline goal files** (`skills/throughline/SKILL.md`) are the executable mission format: a concrete outcome, observed starting state, artifact, proof, authority boundary, rollback, value criterion, realism axis, typed conjecture and a "now" card.
- **Agentic-consensus panels** (`skills/agentic-consensus/SKILL.md`) run a question across several model families (Claude, Gemini, DeepSeek, GLM, OpenCode and others) for design and phase gates. The ledger metamission went through a lateral round and two convergent rounds; LU's design went through four.

- **No blocking asks**: when a decision lacks one right answer, agents use the panels, take the conservative option, or record a named residual, and keep going. Blocking asks are reserved for irreversible steps, deploy-shape trade-offs and credentials.

An observation on cost. These rules work: the week's incidents were found, documented and mostly fixed within hours. They also cost time and words. The repository has 1,290 files under `docs/`, 418 evidence entries and 111 reports, and design rounds of four panels for one station are common. Section 9 lists process weight as a risk to pace.

5. Current state of the codebase

5.1 Size and shape

Counted from `git ls-files` at 7173cc4a0 (2,790 tracked files), excluding lock files, one generated file, and binaries:

Directory	Lines	What is there
`docs/`	425,738	Evidence (163k, mostly JSON receipts), archive (136k), reports (31k), problems (30k), definitions (26k), decks (24k)
`internal/`	369,628	Go: 216,162 source and 146,687 test lines in 86 packages
`frontend/`	58,261	Svelte app (about 36.7k) and Playwright specs (about 19.4k)
`cmd/`	19,384	22 service and tool entry points
`scripts/`	9,958	Probes, tracers, deploy helpers (bash, Node, Python)
`.github/`	4,639	CI (`ci.yml` is 1,805 lines)
`skills/`	2,904	Agent skills (throughline, agentic-consensus, ...)
`nix/`	2,687	Host and guest NixOS configuration
Total	**about 895,000**	

By language: Go 377k lines (227k source, 151k test, in 1,194 files), Markdown 275k, JSON 135k (128k of it evidence), JavaScript 34k (mostly Playwright), Svelte 28k, YAML 15k, TypeScript 4k, Nix 2.8k.

Test code is about two thirds the size of non-test Go code.

The 22 binaries in `cmd/`: `auth`, `autoputer` (the guest daemon), `capsule-broker`, `checkpointd`, `choir` (CLI), `choir-builder`, `choir-rebuild-base`, `choir-receipt-signer`, `choir-updater`, `corpusd`, `desktop` (macOS), `doccheck`, `gateway`, `goproxy`, `lockscope`, `maild`, `maildctl`, `platformrelease`, `projection-compact`, `proxy`, `vmctl`, `zot`.

The largest packages (non-test Go lines):

Package	Lines	Purpose
`internal/agentcore`	35,491	Guest runtime: desk activations, the RLM loop, the reducer, inbox records, source schedules, self-development operations, live WebSocket, planned restart
`internal/store`	33,021	Guest embedded-Dolt store: runs, trajectories, work items, Texture documents and revisions, story opening
`internal/platform`	16,281	corpusd: Store B, publications and front page, event-head CAS, checkpoints, key escrow, file CAS, exports
`internal/textureowner`	12,176	Texture desk owner: prompt bar, Texture API, the canonical writer
`internal/vmctl`	11,218	Computer ownership and routing: resolve, deploy refresh, hibernate, reclaim, cold recovery
`internal/yaegikernel`	7,436	Go-cell interpreter and per-desk `choir.*` module surface
`internal/proxy`	6,961	Auth-gated router, platform routes, publish relay
`internal/vmmanager`	6,294	Firecracker lifecycle: boot, stop, snapshot, balloon
`internal/maild`	5,387	Email transport and mailboxes
`internal/capsule`	5,344	Self-development capsules
`internal/types`	5,215	Shared types: commitments, lifecycle, work items
`internal/autoputer`	4,753	Guest daemon wiring

Package	Lines	Purpose
`internal/gateway`	4,553	Provider gateway, search, Jev, breakers
`internal/auth`	4,408	Passkeys, sessions, JWTs, API keys, recovery
`internal/computerevent`	3,672	Canonical per-computer event chain
`internal/provider`	3,390	LLM provider bridges
`internal/content`	2,868	Content extraction, PDF blocks
`internal/updater`	2,858	Root guest updater (stage, swap, restart, probe)
`internal/objectgraph`	2,695	Object and edge model, kind registry
`internal/projectionbase`	2,673	Projection base install, compaction, recovery
`internal/texturewrite`	2,407	Texture's single write verb and grammar
`internal/toolregistry`	2,085	Tool-calling loop
`internal/computerversion`	2,046	ComputerVersion and CodeRef
`internal/actorruntime`	1,731	Durable actor runtime adapter
`internal/browsercontrol`	1,610	Web Lens browser control

Sources. `internal/sources/registry.json` registers 220 sources (146 RSS, 73 Telegram via web preview, 1 GDELT), embedded in the binary. The live paper schedules a subset (14, plus 6 added on 14 October).

Frontend. Svelte 4 and Vite 5; `frontend/src/lib` holds 75 files. The desktop registers 17 apps plus Texture (`frontend/src/lib/apps/registry.ts`): Files, Web Lens, Mail, Compute Monitor, Pulse, Texture, Autopaper, Podcast, Image, Audio, Video, PDF, EPUB, Slides, Calendar, Source viewer, Engineering Console, Settings. All calls go same-origin through `fetchWithRenewal` (`frontend/src/lib/auth.js`), which renews a session on a 401 once; live updates come over one WebSocket, `/api/ws`.

5.2 Test strategy as practiced

- **Go tests carry CI.** 573 test files, about 151k lines. The two broad packages, `agentcore` and `textureowner`, are split into six shards (`scripts/go-test-runtime-shards`); the rest run through `scripts/go-test-non-runtime-shards`; load-sensitive tests run in a separate lane; the race detector runs on a sample in CI and fully every week (`.github/workflows/race.yml`).
- **Static gates.** A heresy detector that fails on regression (`scripts/check-heresies.sh`), vocabulary and writer-purity checks, decode roots, test hang guards, and a report-only docs truth checker.
- **End-to-end proof is deployed probes, not CI.** None of the 69 Playwright specs runs in CI. The real end-to-end proofs are operator scripts run against choir.news: the tracer (`scripts/reeng_tracer.mjs`, which mints a fresh staging computer, submits a story, starts a crash drill mid-run and writes a trace bundle), the self-development probe (`m11_selfdev_episode_probe.mjs`), vmctl probes, the lose-the-disk proof, and the L2a scoring scripts. Their receipts land in `docs/evidence/`. This matches "staging is acceptance", but it means no end-to-end regression runs automatically on each push. The operating contract says to "highly prefer E2E tests"; the practice is E2E by hand.

5.3 CI and deploy

`.github/workflows/ci.yml` runs on every push to `main`:

1. **plan** classifies the change (docs-only, Go, frontend). On `main` it diffs against the commit **live on staging** (read from the `x-choir-build-commit` header), so a docs push cannot skip deploying queued code (6d6db691).
2. **vet, build, test shards, scale tests, doccheck, heresy detector, vocabulary gates, SBOMs, frontend build**, then an aggregate `check`.
3. **deploy-impact** decides which host services, frontend, host OS, vmctl restart and guest refresh are needed.
4. **deploy-staging**, serialized and never cancelled mid-run: a stale guard, an SSH build on Node B with node-a as remote builder, activation, frontend install (keeping the previous for rollback), vmctl restart at most once, guest refresh as a planned restart (busy computers are deferred), smoke tests, and a deploy receipt in `/var/lib/go-choir/deploy-receipt.json` (failures write rollback pointers).

Deploys went from about 13 minutes to about 2.7 minutes this week with one Go compile for 15 services and builds on node-a (fe5bbcbf, 63c3263c). The site now stays up during deploys (2db4bc02; 417 of 417 probes returned 200 on the verifying deploy), with a small residual of 502s on one deploy (D4).

Staging identity at the time of writing. `https://choir.news/health` reported proxy build `dd47dd84` deployed at 14:16:45 UTC on 14 October, `vmctl_status: ok`. `main` was five commits ahead (7173cc4a0), including the search backoff fix ac5c17db, which was not yet deployed.

5.4 What is healthy

- **The persistent computer substrate.** Lose-the-disk recovery, write-once key escrow, verified checkpoints, crash closure, bounded restarts (L1a), computer tiers, and snapshot hibernation with a 103 ms wake are all proven on staging this week.
- **Authority boundaries are enforced in code**, not only in prompts: Texture is the sole writer (refused at the product API for other desks), research is read-only, messaging is fail-closed, engineering runs only in capsules.
- **Observability without SSH.** Trace export, the tracer, an invariant checker, `/api/runtime/obligations`, `/api/costs` in dollars, and a public `/health` with lifecycle stage timings.
- **Deletion pressure.** About 50,000 dead lines removed in D0; the old ingestion daemon, the old reader, 17 legacy tables and `spawn_agent` gone.
- **Autopaper publishes continuously** from the host store, with immutable published versions, per-version permalinks and logged-out reads that never touch the paper computer.
- **Deploy speed and safety** improved sharply.

5.5 What is fragile (candidly)

1. Search providers. On 14 October, 7 of 9 providers were cooling; 2,287 searches got one provider and 176 got none; Brave failed 54% of its 3,367 calls; Tavily, Parallel, SerpAPI and Serper ran out of credit. Searches per day rose from 101 (9 Oct) to 4,140 (14 Oct, to 14:08 UTC) because streaming search calls every paid provider. The doc's own clustering: this is the fourth search-plane fix in five days, and "the earlier fixes changed which outcomes count, never how a count becomes a bench" (`docs/problems/search-providers-cooling-most-of-the-time-2026-10-14.md`). ac5c17db on `main` reworks the strike model; refills and a cost model are owner decisions. Each metered provider costs roughly $20-40 a day at this volume. A measurement the same afternoon (`docs/reports/search-efficiency-and-reuse-2026-10-14.md`) found that only about 1.5% of returned results are read and none are kept, 27% of page reads fail at the publisher, exact repeats are 3.5% but a third of searches re-ask within ten minutes what the same computer just asked, and corpus search sees only the titles of the newest 1,000

captures. The levers, in order of money: stop paying every provider for every search, keep what was fetched and look there first, and do not repeat a recent search. The first slice, a result cache and in-flight join in the gateway, is on `main` (7250d36bf).

2. Story synthesis (merging). There is no story-merge step in code beyond Jev's intake labels; the old cluster kind is registered but unused. The fixture reached merge recall 1.0, but the first live window reached 0.174: 38 of 46 cross-outlet same-event pairs (for example BBC and Guardian on the same arrest) ended in different stories, and only 13% of 425 captures were attributed to a story (`docs/evidence/l2a-after-window-1-2026-10-14.md`). On the night of 13-14 October, 47 of 50 stories had opened from exactly one capture (`docs/reports/owner-roadmap-rulings-2026-10-14.md`).

3. Story quality on the front page. At 14:20 UTC on 14 October the public front API returned 20 stories with a median of 3 cited sources and about 225 words. Nine of the 20 titles read as desk instructions ("Write the story of Switzerland proposing to ban ...", "Open a story on the Mediaite report ..."). The "Open a story on" form was already named in the LF intent note; the "Write the story of" form is new today. *Hypothesis, not traced:* the task text from management delegation (S2a, deployed in the pre-demo deploy) is being used as the story title. Per the operating contract this needs a problem record and a trace before a fix. A one-paragraph, one-source stub led the front page earlier in the day (`docs/problems/front-page-leads-with-a-one-source-stub-2026-10-14.md`).

4. Serial management and slow models. From 03:20 UTC on 14 October no stories opened for over 30 minutes: one management run held the desk's only turn for 1,680 s (58 calls, 4.1 M input tokens), and 105 of 132 arrivals found management busy. LongCat's latency was 21.8 s at the median and 932 s at the maximum. Then free-model 429s opened the whole provider circuit; 29 research runs failed and stub stories auto-published (`docs/problems/management-serial-turn-blocks-paper-2026-10-14.md`, `docs/problems/free-model-429-opens-provider-circuit-stub-stories-2026-10-14.md`). Fixed or mitigated: LongCat out of the pool, Jev doing support judgements, per-model breakers, a hold on uncited publishes, and S2a delegation.

5. Restarts and updates of busy computers. Pressure reclaim hibernated the live paper on 13 October (silent about 3 hours) because no tier protected it; a planned refresh read as a crash on 14 October and closed all 14 schedules (silent about an hour). Both are fixed (188f98c3, ce59d406), but the underlying question, how an always-busy computer takes updates, is station LU's.

6. Single host capacity. Node B has 31 GiB RAM and a 476 GB disk (374 GB used on 14 October). A global out-of-memory event on 10 October took down choir.news and SSH; the artifact store was growing 12.6 GB a day with GC only dry-running. A user's computer was hibernated under pressure during disk scans and builds and took five minutes to come back (`docs/problems/pressure-reclaim-wedged-a-user-computer-2026-10-14.md`).

7. Cost visibility. Model spend is cheap now mainly because of free and flat-rate routes. Prompt caching across turns is defeated by a timestamp; Jev calls are not counted; the cost of search is not modelled.

8. Lost publishes. The guest's auto-publish queue is held in memory; a publish lost to a crash is not retried (code survey, `internal/agentcore/story_publish.go`). Revisions that cite nothing are held on purpose.

9. Desk kernel friction. Interpreter faults are the largest cross-model error class. Failing cells fell from 19.9% to 13.1% after the cluster fixes on 13 October, but concurrent `choir.*` calls in one cell can still receive each other's results (`docs/problems/parallel-choir-calls-cross-wire-results-2026-10-12.md`).

10. Retired names in persisted identity. "Universal Wire" appears in 26 non-doc files, including the paper computer's VM identity; renaming persisted identifiers is a migration, not a find-and-replace.

11. Doc drift. Several 14 October problem records say a fix is "on a branch, not pushed" when git shows it on `main`; the operating contract describes a separate docs workflow that is in practice a job inside `ci.yml`. Being on `main` also does not mean deployed: the pre-demo hold and then the combined deploy decided what went live.

6. Progress in the last week (7 to 14 October)

6.1 The week in numbers

Measure	7-14 Oct	30 Sep-6 Oct
Commits on `main`	**1,007**	491
Lines inserted / deleted	**+208,644 / -108,180**	+62,184 / -16,021
Of which docs (`docs/**`, `*.md`)	+95,949 / -14,387	
Of which code and other files	+112,695 / -93,793	
Files touched	1,521	
Authors	1 (with agent co-authorship)	1

The repository has 7,535 commits since its first on 10 April 2026; this week is 13% of them. Commits per day: 18 (7 Oct), 48, 200, 194, 107, 77, **227** (13 Oct, Autopaper live) and 136 by 14:17 UTC on 14 Oct. More than half the commits (544, 54%) are `docs`, 219 of them problem records, which reflects the problem-documentation-first rule more than effort. Deletions peaked on 12 October (-60,055 lines) with the D0 dead-code census.

Commit types by mutation class: 103 `red`, 40 `orange`, 30 `yellow`, 94 `fix`, 90 `feat`, 35 `chore` (mostly deletions), 19 `evidence`, 16 `test`. Commit counts in this section come from subject-line grouping and are approximate (about plus or minus 10%).

6.2 What happened, day by day

Day	Focus	Key commits
7 Oct	Management desk cutover finished; owner computer's boot loop fixed	7a36713c5, e9cd9fed0, ca41be90e
8 Oct	Recovery and state homes: verified incremental checkpoints, write-once key escrow; three-gate roadmap written	0f7c58ba6 (+8.6k), c990bf505, 9331de3b6, 2e93ec652
9 Oct	Gate 1 hardening: host memory budget, builds on node-a (deploys 13 min to 2.7 min), Texture zombie fix, "restarts end work" in code, **lose-the-disk proof passes**; World Wire renamed Autopaper	fe5bbcbf, 07e7e0cd2, f6001aea2, 6d1215a02, 677dee8f2
10 Oct	Gate 2: after a Node B out-of-memory event, vmctl capacity guards and "no automatic kill of a live computer"; **first complete supervised self-development episode** (M11 rerun 13, 17 of 17 legs)	cf0969cf, 537735c1, 86770ae7, 17b97b8fe
11 Oct	Autopaper W1: captures in the shared store through corpusd, schedules as durable wakes; **precommitment-record protocol P1-P2** for all inter-agent messages; research tuning stopped for a structural fix	b36d58b3, 0f1c86c7, d1d83c39, f21bd2d0, 5b93ba54
12 Oct	Re-engineering decisions and the "radical cutover" into three cuts; model pools; **about 50,000 lines deleted**; the tracer ("eyes") built	fbb58c23, 2d2b864e, b0d69d4d, 90fb2246, 764279a9, ec15c4b6
13 Oct	Cut 1 (records as work state) accepted; Cut 2: one write verb, per-version permalinks, old ingestion deleted; **Autopaper live**; ledger metamission and restart design v3; computer tiers after the paper was reclaimed	76ab40fd, f93fdcb9, f12ee4be, 03964b06, f124766f, 8a1477c3, 188f98c3
14 Oct	Overnight: C1e B1, cost levers 6-7, balloon, **snapshot hibernation (103 ms wake)**, schedules survive restarts; morning: Go module proxy, station LU, roadmap stations, owner reorders to optimizations first; pre-demo deploy (LF0, S2a, GC, cache metrics); then LF1 and the search-provider fix	a06191f9, 34f3a75f, d07e4cd3, 7ad5607f, ce59d406, 3c6347e8, f4650420, 1f84fa90, ac5c17db

6.3 By station and area

Gate 1, a stable computer (about 40 commits, 7-9 Oct). Verified incremental checkpoints (0f7c58ba6), write-once custodian escrow for privacy keys (c990bf505) and delivery of the escrowed key to a new realization (9331de3b6). The **lose-the-disk proof** passed on staging on 9 October: a disposable computer lost its disk, the next realization got its key from escrow, rehydrated 2,460 files and served a private proof file 67 s later (6d1215a02). The debugfs key-copier backdoor was deleted (a72e2d32a).

L1, restarts (about 66 commits). The owner rule "a crash ends work, a planned update resumes it" went from doc to code for every desk (07e7e0cd2); deploys stopped restarting running computers (f6001aea2); an event-driven wake outbox got a bounded, visible retry budget (b78a8aeb4); obligations became readable without SSH (ee686ab5f). Under the ledger metamission, L1a-1 (start limits on every restarting unit, dc184db2), L1a-2 (bounded self-applies, c08ead07) and L1a-3 (crash boots settle from the updater journal, 18d01cc0) landed, and standing source schedules now survive restarts (ce59d406).

Gate 2, self-development (about 111 commits, mostly evidence, 9-10 Oct). Thirteen reruns of the M11 probe, each fixing one blocker: capsule sandboxing (seccomp and Landlock, e32258209), the apply hold (7ea0f66db), replay-stable ids (f95311775), a patch the freeze proves with git (1ae758ef), Jev judging the frozen candidate (32febcf6). **Rerun 13 satisfied all 17 legs** (17b97b8fe). Capsule network grants (284f7c1a) and a host Go module proxy (3c6347e8, 14 Oct) followed.

vmctl and host capacity (about 53 commits). A Node B out-of-memory event led to host-capacity guards (cf0969cf); a review converged 8 of 8 on a vmctl redesign whose Phase 0 made destruction attributable and removed automatic kills of live computers (d41c2caf, 4eb1ec55, 19ba3034); a wedged computer now recovers by itself (537735c1); a console-pipe deadlock that froze the owner's computer was fixed (86770ae7); **computer tiers** keep paid and platform computers on (188f98c3); reclaim answers only real pressure signals (d3f7d9b2).

Texture (about 55 commits). A Texture acceptance suite, driven green apart from one owner decision (58acdad5); the "Revising forever" zombie loop fixed (1b6c1b732); cold document list from 2.67 s to 0.19 s (c01bc2f9d); boot replay from 4.7 min to 31 s (per `docs/ACTIVE.md`).

Research and persistent desks (about 74 commits). Research fan-out was tried and replaced by one persistent research desk per document (2ac20789); a desk's Go session now outlives its run (fcb424b9); a computer-wide model call gate waits out rate limits (3faa001a).

Precommitment protocol P1-P2 (about 30 commits, 11 Oct). Every Texture control mints a cast record (0f1c86c7); Texture dispatches with Cast, Ask and Note (d1d83c39); the product API refuses Texture writes from every other desk (f5183706); the old Message and Emit channels retired (b196d666).

Re-engineering metamission, L0 (about 200 commits, 12-14 Oct). Three cuts replaced six planned stations (2d2b864e):

- **D0**, about 50,000 lines of dead code deleted with owner approval (90fb2246).
- **C0, "eyes"**: a trace bundle format, `choir trace export`, an invariant checker and a tracer that mints test computers without SSH (764279a9, ec15c4b6, 76ce0b89).
- **C1, authority**: crash boots close pre-boot runs, records as work state, `spawn_agent` and 17 legacy tables deleted (76ab40fd).
- **C2, evidence to publication**: one Texture write verb `choir.Write` (f93fdcb9), per-version permalinks (f12ee4be), the old ingestion daemon deleted (03964b06), the Autopaper reader app (579e35d1), and **Autopaper live on the designated paper computer** (f124766f, 13 Oct).
- **C1e**, engineering on records: B0 (19e56c54), B1 (a06191f9, deployed e9d727a7), capsule-gate and Freeze fixes.

L2a, synthesis intake (about 34 commits). Jev labels each new capture against a story shortlist (ba297cbd); a per-computer `intake.labels` setting (d6f82142); a lossless per-poll limit and durable pending-arrival list (969ee109, 27efc950). Fixture run: merge recall 1.0, no false merges (a9d87870). **Live window: merge recall 0.174** (8 of 46 cross-outlet same-event pairs merged; 1421dde3).

LF, front page (about 18 commits). A broadsheet front page (3e81975c); LF0 importance scores from Jev with recency decay (e6291f9d, e6d9215d); rank by importance times depth times recency, no one-source lead, and four sections (1f84fa90).

LR, resource and cost (about 36 commits). Memory assessment (b5aec925); lever 6 takes bulk cell output out of the transcript (34f3a75f); lever 7 puts model prices in the cost record, so `/api/costs` reports dollars (d07e4cd3); LongCat out of the default pool for latency (3ee85469); virtio balloon with free-page reporting (0b28b9ec) and a soft Go memory limit (0c23665b); cached-token measurement (34a3e524); the artifact store leak of about 12 GB a day found and fixed (89103722), and garbage collection activated with a seven-day window (fa3b3336).

S3, snapshot hibernation (about 11 commits, 14 Oct). Hibernate takes 6 s; wake to healthy in 103 ms against 9.7 s cold, with open work kept (ab701e32, 7ad5607f); a 55-minute hibernation past the credential deadline resumed in 396 ms (7ca4a55b).

S2a, management delegation (about 17 commits, 14 Oct). Management can delegate intake tasks to sub-RLMs that run and close on records (a2d3d1f8). On the paper every task was first refused by the vocabulary fence; fixed in 6f8e36ce.

Search (about 21 commits). A long series of fixes for providers spending most of their time in cooldown, ending today with "one episode, one strike", bounded backoff and pacing Brave instead of benching it (ac5c17db). Four paid accounts are spent and need the owner (667428be).

Deploy and CI (about 22 commits). One Go compile for 15 services and builds on node-a (deploys 13 to 2.7 minutes; fe5bbcbf, 63c3263c); CI plans against the live staging commit (6d6db691); deploys keep the site up while the gateway drains, ending an 11-minute outage per deploy (2db4bc02).

Roadmap and research (docs). Station LU (guest updates) and four panel rounds; LP, LA, SP, LT, LF, LC added to the metamission; the multilingual OSINT and Taiwan source research; the HeyPuter app review; the media and rights research; the October 9 revision of the prospective self-alignment paper (428b79022); the pre-seed deck.

6.4 Problems and incidents: what kept breaking

The week produced **119 problem records** and **76 evidence entries** (35 structured acceptance receipts, 15 M11 self-development reruns, 9 Texture acceptance runs, 7 management-desk receipts). Grouped by root cause:

Cluster	Records	Typical problem	State
Desks, Texture, kernel	about 20	Obligations that close only when the model chooses; zombie activations; verb errors vanishing; cells cross-wiring results	Mostly fixed; protocol friction open
Self-development and capsules	13	Seccomp killing the worker; freeze on new directories; checkpoint starved by resumed work; Go modules	Most fixed through M11; egress decided (host module proxy)
Deploy and CI	13	Deploys restarting busy computers; an 11-minute outage per deploy; health gates not bound to the commit	Mostly fixed
Research desk	12	Reports rejected by schema; research stopping after one report; about 1% of search results cited; fan-out overrunning budgets	Structural fix (one persistent research desk per document); several open
VM pressure, memory, disk	10	Global OOM; 12.6 GB a day of artifacts; reclaim hibernating the paper and a user	Partly fixed

Cluster	Records	Typical problem	State
Management and delegation	9	Serial turn blocking the paper; casts reaped before spawn or dying with their turn; S2a tasks refused	Partly fixed
Restarts, boot, crash	7	Crash closure covering only wakes; crash boots re-applying; planned refresh read as a crash	Mostly fixed (L1a, ce59d406)
Autopaper stories	7	46 of 118 arrivals unanswered (cursor bug); markdown in prose; intake defects; one-source stub lead	Partly fixed
Search providers	6	A cancel, an empty result or one 429 benching providers for minutes to days	Fourth fix pushed today
Storage, Dolt, recovery	6	423,720-event recovery tail; whole-chain replay scans; GC holding a write lock	Mostly fixed
Models and inference	4	One computer's 400s opening a breaker for all; duplicated tool calls	Partly fixed
Sources, auth, frontend	about 12	Silent poll failures; escrow overwritable by a guest (fixed); recovery never sending its token	Mixed

The recurring substrate patterns, in the docs' own clustering assessments:

1. **Obligations with no driver and no terminal fate.** Texture activations, research items, delegated casts, the records cursor and cancelled-run rounds all failed the same way: work that nothing would ever finish or close. The 9 October obligation inventory found one periodic driver against about 30 boot or event re-drivers, with retry budgets on only four paths (`docs/evidence/sl-obligation-inventory-2026-10-09.md`). The "records as work state" cuts (C1, C1e) and the restart rules are the substrate answer.
2. **Health counters that punish the wrong party.** Caller cancels, client errors and empty results were counted as provider failures, in search (three times) and in the inference breaker.
3. **Restart semantics.** Crash versus planned, deploys versus busy computers, reclaim versus protected computers, and an epoch bump that made a planned restart look like a crash.
4. **Authority anchored on turn-scoped facts.** Five delegated-cast defects in one day came from tying a cast's authority to the turn that made it, not to a durable object.
5. **One host.** Every capacity incident traced back to one machine with no declared memory or disk budget until this week.

The eight incidents that mattered most

#	Incident	Effect	Root cause	Fix
1	Search providers benched most of the time (14 Oct)	Thin research; a one-source stub led the front page	Per-call strikes with unbounded backoff; no cost model	97970cda, 4d365e47, b7668dc3, ac5c17db; owner decisions open
2	The paper stalled behind management (14 Oct, from 03:20 UTC)	No stories opened for 30+ minutes; 28 captures lost; stubs published	Serial management; LongCat tail latency; one breaker per provider; no citation check on auto-publish	3ee85469, d5139fd6, bde0a169, e3437f1c, S2a
3	Restarts silenced the paper (13-14 Oct)	About 3 h and then about 1 h without publishing	No computer tiers; epoch bump made a planned refresh look like a crash	188f98c3, ce59d406
4	Node B out of memory (10 Oct)	choir.news and SSH down; owner computer rebooted twice	vmctl did not track all VM processes; no memory budget	cf0969cf, 16806d8e
5	Deploys harmed what they deployed (9-14 Oct)	Busy computers restarted; 3-11 min outage per deploy	Restart paths not gated on busyness; gateway drain on the request path	f6001aea, 2db4bc02, 3439462ab

#	Incident	Effect	Root cause	Fix
6	Texture zombie activations (9 Oct)	Owner's boot took 9-11 minutes; 14.7 GB RSS	68 runs re-armed every boot with nothing to execute them	3083c6dd and related; boot to 31 s
7	Delegation never reached engineering (10 and 14 Oct)	Engineering casts cancelled on arrival	Turn-scoped authority; then Go module egress	bc09dcc1; host module proxy
8	Firecracker console deadlock (10 Oct)	Owner's computer froze	SIGPIPE on the console after vmctl exited	86770ae7

What the week taught.

- Speed of response is high: most incidents went from problem record to deployed fix within hours, with evidence.
- The fixes are increasingly substrate-level (records as work state, tiers, bounded restarts, per-model breakers), which is what the "convergence before patching" rule asks for. Search is the exception: four symptom fixes in five days before the strike model itself was changed.
- Free models and serial desks interact badly: a slow free model's tail latency became product downtime.
- A fixture result does not predict the live result: L2a went from recall 1.0 on the fixture to 0.174 live.

7. Expected progress

This section is a forecast built from the roadmap documents (`docs/ACTIVE.md`, the ledger metamission, `docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`, `docs/reports/owner-roadmap-rulings-2026-10-14.md`) and the pace of the last two weeks. It is not a commitment. Each item names its station, its dependencies, and the decision it needs from the owner, if any.

7.1 The order of work, as the owner set it on 14 October

```
now ........ OPTIMIZATIONS (LR + S3 + SA): RAM, disk, tokens per task
             |  "we really shouldn't be scaling our utilization until we
             |   improve our efficiency to the point of diminishing
             |   marginal returns" (owner, ~12:00 UTC)
             v
then ....... AUTOPAPER SCALING: L2a synthesis on, management delegation,
             research corpus-first, deepen important stories, more sources
             |      (in parallel: LU guest updates, L1 restarts, C1e B2-B5)
             v
then ....... LA auth (small, early)  ->  Gate 2 self-development (S1, S4,
             S5, S6)  ->  LP app packages (S7-0: existing apps first)
             v
later ...... L3 hypotheses settled and scored -> L4 Taiwan vertical
             -> L5 evidence reader + Autoradio prototype
             S8-S11 (publication/adoption, forks and fleets, org
             templates, fleet pushes), SP infrastructure, LT red teaming
             LF personalization and LC contributions after design review
```

Data-model work on the object graph and Dolt runs throughout ("always").

7.2 Next week (15 to 21 October)

Optimizations (station LR, with S3 and SA pulled in). The owner's list, in order of expected effect:

Item	Why	State on 14 Oct
Artifact store leak	About 12 GB a day of growth on Node B	Fix 89103722; GC activated with a 7-day window (fa3b3336); measure the reclaim
Snapshot budget by real bytes	Hibernated computers keep deleted-but-mapped snapshot files	Deployed in the pre-demo deploy (ee2267ba)
Prompt-cache hits	Every desk prompt carries the time to the second at about token 275, so nothing after it is cached across turns	Measurement landed (34a3e524); the fix (move or coarsen the timestamp) is next
Guest daemon's store memory	690 MiB private in the daemon vs about 17 MiB per desk worker: "the real density lever"	Measured (`docs/evidence/guest-memory-split-balloon-2026-10-14.md`)
Guest memory right-sizing, KSM	More computers per host	Design only
Disk cleanup, Nix store retention	Single host disk pressure	Nix GC timer deployed (51c522b1)
Jev cost accounting	Jev calls are missing from `/api/costs`	Gap named in `docs/evidence/model-prices-2026-10-14.md`

Expected result by the end of the week: a measured per-computer RAM and disk figure after the levers, a measured cache-hit rate, and the oversubscription probe (how many paper-sized and small computers fit on Node B). The provisional read is about 4-5 paper-sized or about 20 small computers before pressure (`docs/reports/lr-scaling-cost-provisional-2026-10-14.md`).

Autopaper scaling (L2a, S2a, LF), after or beside the optimizations.

- L2a: adjudicate the 31 false-merge candidates and trace the 38 missed pairs from the live window; then raise live merge recall well above 0.174 without false merges. This is the single most important quality measure for the paper, because the owner's product principle is "one story per real-world event".
- S2a: management delegation to sub-RLMs, re-measured on the paper after the vocabulary-fence fix; it attacks the "management serial turn blocks the paper" problem.
- Research searches the corpus first, then the web; deepen important thin stories (design note, ships dark behind `paper.deepen`, about three deepenings an hour, provider health first).
- Search providers: the backoff fix (ac5c17db) to deploy and measure; four spent paid accounts are an owner action.
- Sources: the eight U.S. politics and business feeds are scheduled after deploy. The target measure is sublinear article growth: sources per story and captures per story should rise while stories per hour rise far less than captures per hour.
- Fix instruction-phrased story titles (section 5.5).
- Search reuse (`docs/reports/search-efficiency-and-reuse-2026-10-14.md`): measure the result cache (7250d36bf), then keep every search and page read as objects with provenance, look in held text first, and route by budget instead of calling every provider.
- Claims, events and entities (design): the offline study on a frozen, blind set and the say-do eval of Choir's own desks, which can stop the layer before any production code.

Guest updates (LU). LU1a (host-only retry) and LU2 (persisted epoch and migrations) are cleared to start; LU2 must land before C1e's next epoch bump. LU1b (the drain's final call) awaits a structural decision after four review rounds. This matters because the paper is never idle and today takes updates only by an owner act.

Restarts (L1). The fault-injection harness and L1b (a boot ledger: one automatic retry per external event, then a visible hold).

Closing L0. C1e B2-B5 and the re-engineering final review. The engineering drill should now get past the Go-module wall through the host module proxy (3c6347e8, fixed in 3439462ab); whether it reaches Freeze is the next observation.

Owner decisions likely needed next week: the remaining LU questions (O1-O3b, O5, O7-O10), the 12-hour snapshot maximum age, SP0's host and recovery targets, paid search accounts, and whether LA comes before self-development (the sequencing proposal says yes).

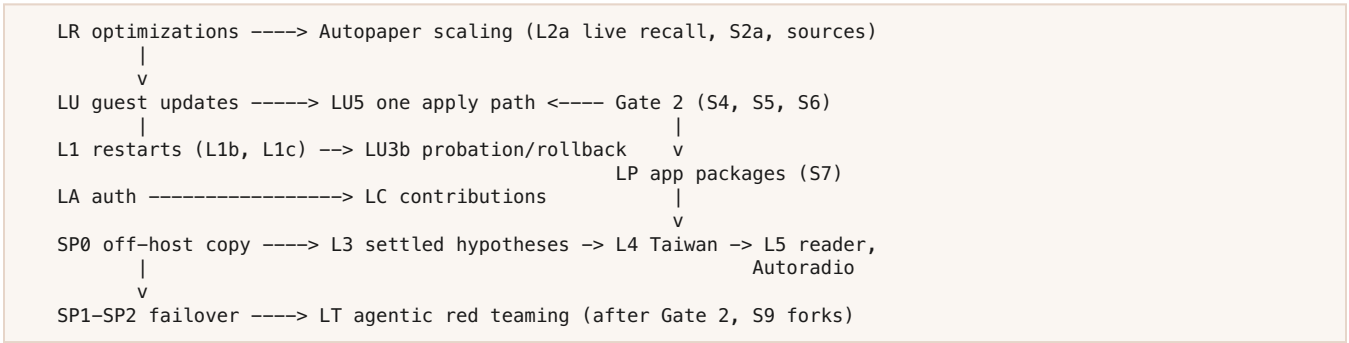
7.3 Next month (to mid-November)

Station	What lands	Depends on	Risk
LA, auth	Email magic-link sign-in through maild, a working recovery screen, several passkeys, visible sessions	none	Today a user who loses their passkey cannot get back in: the recovery token is created and discarded (`internal/auth/recovery.go`, about line 152)
Gate 2, self-development	S1 security-floor remainder, S4 open-world capsule, S5 live preview, S6 commit gate; an owner-requested change built, previewed, approved, released and restored on the records substrate	C1e, the Go module proxy, LU5 (one apply path)	The substrate moved under the October 10 proof; reruns will find new layers
LP, app packages	S7-0: existing first-party apps converted to packages, fetched from a host cache	S6, L2a, LR, LU	Defining the generic app-document kind and verb registry well

Station	What lands	Depends on	Risk
SP0-lite	Encrypted off-host backups and a log/metric sink on part of node-a	owner go-ahead and the other project's agent	Choir data on a host another project administers must stay encrypted
L2b	Digest, triage and burst bounds; every capture gets a recorded disposition	L0	Cost per capture as sources grow
Local embeddings	EmbeddingGemma 2 on Node B under a 3-core idle-priority quota; backfill on a rented GPU (about $10) or by API (about $18) after a parity test (`docs/research/embeddinggemma-2-cpu-2026-10-14.md`)	the embedding contract pinned first	CPU contention with guests; a GPU only pays off for multimodal or 10-50x volume
L3, hypotheses settled and scored	Hypotheses with `resolves_by` and criteria; separate settlement and scoring records; Brier per desk as a view	L0 and an off-host copy of the ledger	Goodharting: vague or near-certain hypotheses
L4, Taiwan	10-20 standing questions with scheduled settlements (TWSE, CEC, KEV); about 66 East Asian feeds	L2, L3	Licences (CNA and UDN are non-commercial, so out)
LF beyond LF1	Scored importance predictions; personalization on the reader's computer	L2a, design review	Do not change the live front page without owner review
LC	Op-eds, suggested edits, bring-your-own sources	LA, design review	Moderation and labelling
L5	Evidence reader; an Autoradio demo framed as a prototype	L2, L3, L4	"No slop" quality bar

A realistic reading of the pace: the last week landed about one station's worth of deployed proof per day while also absorbing incidents. If that holds, the next month can plausibly close L0, L1 and LU, land LA and most of LR, and bring Gate 2 back to a complete episode on the new substrate. L3 and L4 are the first stations that make the "dated, resolved ledger" real, and they depend on the off-host copy, which depends on SP0 or SP0-lite. L5 and Autoradio are unlikely before late November on the owner's own ordering.

7.4 Dependencies that decide the schedule



The two items that most constrain the month are the **off-host copy of the ledger** (it gates L3, and every durable home today lives on one host) and **LU's drain decision** (it gates safe updates to the always-busy paper and, through LU5, the shared apply path with self-development).

8. Business model and the medium-to-long-term vision

This section records the owner's business model as stated on 14 October (about 15:05 UTC) and sets it beside what the repository has today. Almost all of it is **vision**: no billing, pricing, data feed, or customer exists in the code. Where a design or a built piece supports it, the section says so. The owner's words are quoted where they carry the decision.

8.1 The owner's statement, structured

> "After we do all the optimizations, the compute and VMs will become a de minimis cost relative to model inference. And Autopaper has regular software economies of scale on inference because we can amortize across our user base. So really the per-user dynamic costs are about research and engineering and analysis inference that the user themselves, or rather their agents and data system, are requesting. And because that would inherently be serving real demand, we can realistically charge for it."

The rest of the statement sets the pricing rules (8.3), the products and customers (8.4), and the long-term vision of an agent social network and agentic consulting (8.5). It ends with open questions the owner named himself (8.6).

8.2 Cost structure

Cost	Fixed or per user	What drives it	What we know today
Autopaper inference (polling triage, research, writing, citation checks, ranking)	**Fixed per paper, amortized across all readers**	Number of sources and events, not readers	A priced window on the paper cost 0.33−0.65 in usage, 2.91−3.23 at list rates, 81% of list cost on a free-for-now model (`docs/evidence/model-prices-2026-10-14.md`). At list rates a 14-source paper is estimated at about $109 a day (`docs/reports/vm-density-limits-2026-10-14.md`). Logged-out readers trigger no inference: reads come from the host store
Search (paid web search providers)	Mostly fixed for Autopaper; per user for personal research	Searches times providers per search	About 4,000 searches a day, each fanned out to every healthy provider; roughly $20-40 a day per metered provider at that volume; about 1.5% of results are read and none kept (`docs/reports/search-efficiency-and-reuse-2026-10-14.md`)
Embeddings (planned, for corpus-first retrieval)	Fixed	Tokens of captured text	22-45 M tokens a day; fits Node B under a 3-core quota; a backfill is about $10 on a rented GPU or about $18 by API; an API serves the same model at $0.02 per million tokens (`docs/research/embeddinggemma-2-cpu-2026-10-14.md`)
Personal inference (a user's own research, engineering, analysis; Autoradio conversations)	**Per user, on demand**	What the user and their agents ask for	Recorded per call with list prices and cached tokens, shown per computer at `/api/costs` (`internal/llmcost`, `internal/apihandler`); not billed
Compute (the VM, memory, disk)	Per user (a computer each), plus the paper and platform	RAM held, disk held, CPU while working	A paper-sized computer holds about 5.6 GiB host RSS; small computers 1.2-1.3 GiB; Node B has 31 GiB and a 476 GB disk. The density report (`docs/reports/vm-density-limits-2026-10-14.md`, estimates to be replaced by a benchmark) puts a 16-core, 128 GB EPYC 7371 host at about 20 busy computers (CPU binds) and about 1,200-2,000 registered users, with idle computers hibernated to disk
Platform (host, staff, licences)	Fixed		One host today; no paid source licences yet

The owner's thesis on cost: once LR's optimizations land, compute becomes a small cost next to inference. That is plausible but not yet measured. Evidence pointing each way:

- *For:* the density report's arithmetic. Twenty paper-like busy computers would cost about $2,200 a day in inference at list rates, very likely far more than the daily cost of the one host that runs them (host prices are not recorded in the repo), and model account rate limits bind before the host's CPU does. Snapshot hibernation frees a sleeping computer's RAM entirely and wakes it in 103 ms; a desk worker costs about 17 MiB of private memory; most of a computer's RAM is the store and its caches, which LR targets.
- *Against, for now:* the paper's inference is cheap mainly because of free and flat-rate routes (a free model and a ChatGPT subscription); at list rates and full cache misses, inference per story is higher than the current bill suggests, and search is a larger and less visible cost than either.

Amortization in Autopaper. The paper's spend scales with the world (sources and events), not with readers. Reading a published story costs serving bandwidth, not new inference. That is the "regular software economies of scale" the owner describes, and the architecture already enforces it: published reads go to the host store and never touch the paper computer (section 3.8). The same holds for the planned box scores (8.4): a view over settled records, computed once and served to many.

8.3 Pricing principles (owner, 14 October)

Principle	Owner's words	Status in the repo
Zero margin on inference	"We would not be extracting any margin whatsoever on inference ... we'll sell inference at zero margin"	**Vision.** Costs are metered per call at list price (`internal/llmcost/pricing.go`), which is the prerequisite; there is no billing
Bring your own inference	"Have ways for people to plug in their own inference if they already have inference credits ... we will support that eventually"	**Vision.** Today the gateway holds *platform* credentials only (`internal/provider/provider.go`; providers register when the operator's credentials exist). The ChatGPT route is the operator's own ChatGPT subscription through Codex OAuth (`ChatGPTProvider`, `https://chatgpt.com/backend-api/codex/responses`), shared by every computer. There is no per-user provider credential, no per-user routing, and no sign-in with a model provider. The LA scope note records "an OpenAI sign-in for users' own credits" as a later idea and that the owner does not want Choir reselling ChatGPT (`docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`)
Bulk and self-hosted inference	"Buy inference in bulk and offer it at a lower price than API rates. And even self-host some of our own inference"	**Vision**, with one related research note: self-hosting embeddings is sized and recommended (CPU now, GPU only for multimodal or 10-50x volume)
Margin on compute	"We will charge margins for cloud compute for VMs, for memory and storage"	**Vision.** Computer tiers exist (paid accounts always on, the platform tier above; 188f98c3), which is the product shape a compute price would attach to
Free reading, paid generation	(seed deck, 8 Oct) "Reading a shared result can be free. Asking Choir to produce a new, personalized result is paid"	Consistent with the architecture; no paywall exists
No paid placement in reputation	(seed deck)	A design rule; no ranking product exists

How the pieces fit. Inference at cost makes Choir's price for thinking the same as the provider's, so the user has no reason to route around it; compute and storage carry the margin because they are where Choir's engineering creates the efficiency. Bring-your-own inference then costs Choir nothing, and bulk or self-hosted inference becomes a discount that the user sees directly. One consequence to design for: if a user's inference is at cost, the paper's amortized inference has to be paid by something else (API and data sales, subscriptions, compute margin).

Two things to check before building bring-your-own.

1. Provider terms: whether a consumer subscription (ChatGPT, others) may be used to power a third-party, multi-user service, both for users' own credits and for the operator's current ChatGPT route. This report did not check the terms.
2. Security: per-user provider credentials would live in the gateway beside platform keys; the gateway is a red-class surface and the only credential holder by design.

An earlier price sketch. The 8 October seed-deck treatment proposed three metered prosumer tiers ($35, $100 and $350 a month), then enterprise plans, data licensing and net commissions on third-party agent transactions (`docs/deck/choir-seed-deck-treatment-2026-10-08.md`). The 14 October statement refines it: subscriptions index *time spent*, which indexes inference demand, with inference passed through at cost.

8.4 Products and customers

Product	Customer	What it is	Status
Autopaper (public site)	Everyone	The free, logged-out newspaper	**Built, live**
Autopaper API and data feeds	Funds first ("just to get some cash"), then others	Structured access to stories, claims, events and settlements; the Taiwan vertical as the first saleable feed	**Vision.** The front-page API is public and read-only today; there is no authenticated data API, rate plan or feed. The Taiwan vertical (L4) is intent
Autoradio subscription	Consumers and prosumers	Personalized listening and voice; tiers by time spent, which indexes inference demand	**Vision** (section 2.11)
The box score (aggregate statistics)	Media, funds, institutions, the public	Track records per speaker, outlet, organization, agent and for the platform and the world	**Designed** in the claims-events-entities ontology; no code
Agent API	External agents	Programmatic access for agents to the paper, the box scores and Choir computers	**Partly built**: scoped API keys and the CLI operate a computer today; no published, versioned contract and no metering per key
Compute (the computer itself)	Prosumers, then organizations	A persistent computer with margin on VM, memory and storage	**Built as a product object**, not as a priced product
Private Choir Clouds	Organizations	A customer's own hosts, platform computers and user computers (section 2.11)	**Designed (ontology)**

The Taiwan vertical for funds. The owner plans to go to market with an Autopaper API "as soon as we really dial in the [Taiwan] vertical". L4's design is a good fit for that customer: standing questions with scheduled, structured settlements (Taiwan stock exchange monthly revenue, election commission data, ministry of defence figures, CISA's exploited vulnerabilities list), about 66 East Asian feeds, and a dated record of what was said before it settled. What a fund would buy is *earliness and calibration*: claims captured at a provable time, settled against structured outcomes, with lead times per source. Dependencies: L2 bounded attention, L3 settlement and scoring, L4 itself, and an off-host copy of the ledger. None has landed; on the owner's ordering, L3 and L4 follow the efficiency and scaling work, so a saleable Taiwan feed is a matter of months, not weeks.

The box score: the big data product. The owner calls the aggregate statistics "the real big data product ... not just for individuals, but for the whole platform, the whole world, everyone". Today's design (`docs/reports/claims-events-entities-ontology-2026-10-14.md`) makes it concrete:

- A **claim** is one attributed assertion (speaker S said Y at time T, anchored to a quote in one capture). It is the precommitment Choir records on the speaker's behalf, with a provable capture time. Only claims from scheduled, timely captures enter a track record, so nobody can cherry-pick.
- Claims are **typed by how they settle**: facts by later evidence, forecasts at a date with proper scoring rules, commitments by observed actions, stated values by consistency of later actions, and Choir's own agents' precommitments by the trace.
- **Five uncertainties** are recorded on every claim (attribution, interpretation, the speaker's own confidence, extraction, settlement) and carried into every score; unscorable statements are shown as unscorable, never as passes or fails.
- **Say-do alignment** ("can I take you at your word?") is the share of an actor's checkable commitments that its observed actions kept, with sample size and uncertainty. The owner calls reputation in this sense "the deep compression of intelligence", for humans, organizations, APIs and AI agents alike.
- A **box score** is a view, never a stored number: scope, window, topics and a scoring policy over settlement records. It never shows a single overall trust number or a rank of people, and below its sample bar a cell reads "n too small". Choir's own paper is scored publicly like anyone else.
- **Order:** an offline study first, with the say-do eval of Choir's own desks beside it; public commitments and say-do come eighth of nine slices.

Media: sell or give away? The owner's open question: sell the box score to media organizations, or give it to a broadcaster "if they put the number on TV, which is very valuable". Giving it away for attribution is a distribution play: a number on screen with the Choir name builds the brand and the habit of asking "can I take them at their word?". The design's rules matter more here than anywhere: snapshots pinned with their query and policy, the scorer named, never a characterization of a person, and nothing below the sample bar. A televised number that later proves wrong would be the most expensive kind of error.

Government intelligence. Listed by the owner as an open question, with the note that some customers are "more exciting to me than others". This report reads that as mixed enthusiasm, not a plan. See the neutrality risk in 8.6.

8.5 The medium-to-long-term vision: an agent social network and agentic consulting

The owner: "I'm really interested in how we can use this to eventually build a social network for humans and agents, and a vision of consulting where one agent, in doing a task, subcontracts to dozens or hundreds of other agents which provide signals based on their proprietary data and insights without leaking that data ... Insofar as you can learn more from a query than you leak in giving the answer, an agentic consultant can generate alpha as it's working."

The social network. People and agents are both participants: each has a computer, publishes into the Autopaper object graph, and carries a box score built from what it said and what then happened. Following, replying and op-eds (station LC) are the human-facing part; agents querying and paying each other are the agent-facing part. Both rest on the same record: what you committed to before the evidence, and how it settled.

Agentic consulting.

```
a task arrives at agent A (a user's research desk, a fund's analyst agent)
  |
  | A decomposes it and asks: who has signal on this?
  |   (box scores by topic: calibration, say-do, lead time)
  v
A sends questions to dozens of specialist agents B1..Bn
  each question is a precommitment by A ("I expect X; this resolves when ...")
  each answer is a precommitment by Bi (a probability, a signal, a confidence)
  |
  | answers flow back; Bi's private data never leaves Bi's computer
  v
A combines the signals (weighted by each Bi's track record), acts, reports
  |
  | later: the outcome settles every record
  v
scores update: A's and each Bi's box scores; payments can depend on them
```

The leak-versus-learn trade-off. Every query reveals something about the asker (what it is working on) and every answer reveals something about the answerer (what its data says). The owner's claim is that an agent that learns more from the questions it is asked than it leaks in its answers gains information ("alpha") by working. Three things make this a market rather than a leak:

1. **Precommitment records** give both sides a dated, falsifiable record of what was asked and answered, so a specialist cannot claim afterwards to have known, and a buyer cannot quietly ignore a correct answer.
2. **Reputation as price.** A specialist's settled track record on a topic sets what its answer is worth. The PSA paper's hypothesis H4 is exactly this: a temporally prior, domain-specific record predicts later performance better than popularity or ratings (`docs/prospective-self-alignment-2026-10-09.md` §8). It is untested.
3. **Authority and boundaries.** Answers flow between computers only by publication or an authorized API call, never by direct access to another computer's data (`CLAUDE.md`, "Safety": results return to another computer "only through publication and adoption"). Leakage controls (rate limits, query auditing, aggregate-only answers) are the PSA paper's named necessities, "not guaranteed consequences" of the mechanism (§7.2).

Status: vision. What exists: computers that can be driven by API keys, a typed commitment record, a publication path, and Jev as a cheap scorer. What is missing: inter-computer queries, payments, per-topic box scores, and any measurement of leakage.

How it connects to the rest. The same mechanism runs at three scales: Choir's own desks committing before they act (built in part), Autopaper recording what public actors said before the evidence (designed), and independent agents trading signals under the same records (vision). The first is the proving ground for the other two, which is why the ontology design puts the say-do eval of Choir's own desks first.

8.6 Open questions and risks

The owner's own open questions.

1. Freemium for some products? The architecture already makes reading free to serve; the question is which generation to give away.
2. An API for agents to consume? Partly built (API keys); the product is a contract, metering and pricing.
3. Media: sell, or give away for on-air attribution?
4. Funds: the first cash customer, via the Taiwan vertical.
5. Government intelligence: open, with mixed enthusiasm.

Risks the business model adds.

Risk	Why	Link
Licensing for resale of derived data	Selling a data feed is redistribution. Many sources forbid commercial use or redistribution (ACLED without a corporate licence, openrss.org, CNA and UDN as non-commercial, ransomware.live without written approval); Cofacts is CC BY-SA (share-alike on redistribution); terms for some paid news APIs are not stated. Whether a derived statistic (a box score, an aggregate) is "adapted material" is a question for counsel	`docs/reports/multilingual-osint-sourcing-research-2026-10-13.md`; `docs/reports/owner-roadmap-rulings-2026-10-14.md` ruling 12; `docs/research/autopaper-media-and-rights-2026-10-14/README.md`
Substitution	The rights research finds liability where output substitutes for the source. A feed of story text substitutes more than a feed of claims, events and settlements with links	Media and rights README, bottom line 2
EU press publishers' right and AI Act labelling	Applies to what the API serves into the EU as much as to the site	Media and rights README, points 3-4
Neutrality vs customers	Selling to funds or governments creates a perceived (or real) interest in what the paper says and how actors score. The ontology's excellence bar (attribution evidence, several readings, neutrality, right of reply, visible uncertainty) and a public scoring policy are the defence; customer data must never shape scores	Ontology §7.3; owner ruling of 14 Oct on accepted legal risk
Scoring public actors	The owner has accepted the legal risk (defamation) of scoring public statements, "a serious best effort aiming for excellence"; the quality standards stay as the bar. Selling those scores raises the stakes	`docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md` (ruling about 14:55 UTC)
Privacy	Autoradio and personal computers hold private work, mail and voice; aggregate box scores must never be computed over private data; agent consulting must prove that private data does not leave a computer. Dolt keeps history forever, which conflicts with erasure	Section 9.3 (erasure vs versioned history)
Zero inference margin	Leaves no buffer if providers raise prices or free tiers end; the paper's own inference must be funded elsewhere	8.2
Bring-your-own terms	Consumer subscriptions may not permit third-party, multi-user use; the current operator ChatGPT route should be checked too	8.3
Market for signals	Collusion, wash trading of reputations, and sanitized predictions under monitoring are the PSA paper's named manipulation risks (H5)	PSA paper §8
Sequencing	Every revenue line here depends on stations not yet built (L3, L4, LA, LC, an off-host ledger copy); the fastest path to cash (a Taiwan feed for funds) runs through L2-L4	Section 7

9. Risks and open questions

Ranked roughly by how likely they are to hurt in the next month, within each group.

9.1 Technical

Risk	Why it matters	Evidence	Mitigation in the plan
One host	Every durable home (Store A, Store B, content store, escrow, every computer) is on Node B. A disk or host failure loses the platform and possibly the only copy of the ledger	`docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`; platform keys copied offline only on 14 Oct (`docs/evidence/platform-key-backup-2026-10-14.md`)	SP0-lite encrypted backups on node-a; SP with a second host
Synthesis quality	The product principle is one story per event; live merge recall is 0.174 and many stories still open from one capture	`docs/evidence/l2a-after-window-1-2026-10-14.md`; owner rulings observation (47 of 50 stories from one capture)	Adjudicate misses; L2a enable; S2a; corpus-first research
Search providers	Deepening and research depend on web search; 7-8 of 9 providers were cooling in every sampled run	`docs/problems/search-providers-cooling-most-of-the-time-2026-10-14.md`; ac5c17db	Backoff fix; paid accounts (owner)
Instruction text leaking into published titles	At 14:20 UTC, 9 of the 20 front-page stories had titles that read as desk instructions ("Write the story of ...", "Open a story on ...")	Public front API, 14 Oct 14:20 UTC; "Open a story on" already named in the LF intent note	Needs a problem record and a trace before a fix (hypothesis: management task text used as title after S2a)
Lost publishes	The guest's auto-publish queue is in memory; a publish lost to a crash is never retried, so a revision can exist without its public version	Code survey (`internal/agentcore/story_publish.go`)	None yet; candidate for the records substrate
Updates to busy computers	The paper is never idle, so it is never refreshed automatically; one refresh closed its schedules for about an hour	`docs/problems/planned-refresh-closes-paper-schedules-2026-10-14.md`; ce59d406	Station LU (pull with a bounded drain)
Pressure reclaim	Reclaim hibernated a real user's computer during disk scans and builds; the snapshot was rejected after a deploy and the cold boot wedged for 5 minutes	`docs/problems/pressure-reclaim-wedged-a-user-computer-2026-10-14.md`	Signal-specific reclaim (d3f7d9b2), tiers, S3 snapshots
Self-development substrate drift	The October 10 complete episode was on the pre-C1e substrate; engineering's work state has since moved to records	C1e B1 problem record; Go module decision	Rerun the drill end to end after the module proxy; LU5 one apply path
Serial bottlenecks	One management desk turn at a time blocks the paper; the guest's embedded Dolt has historically run on one connection	`docs/problems/management-serial-turn-blocks-paper-2026-10-14.md`; `docs/autopaper-sw-design-2026-10-09.md` §0 (C1)	S2a delegation; SA store split
Deploy safety	Each deploy to the shared guest runtime is still a careful, one-at-a-time act; residual 502s on deploy (D4)	`docs/problems/deploy-takes-staging-down-during-gateway-drain-2026-10-14.md`	2db4bc02; LU; SP1 zero-downtime deploys
Breadth of the code	The two largest packages, `internal/agentcore` and `internal/textureowner`, are so large CI shards them; most incidents this week were in the boot, restart and lifecycle paths	Code survey (section 5)	D0-style deletions; the tracer and invariant checker

9.2 Cost

- **Spend is cheap today because of free and flat-rate routes.** In the last priced window the paper cost $0.33 to $0.65 in usage against $2.91 to $3.23 at list rates, and about 81% of the list-rate cost was LongCat, which is "free for a limited time" (`docs/evidence/model-prices-2026-10-14.md`). LongCat has since left the default pool for latency. When free tiers end, costs could rise severalfold at the same volume.
- **Input dominates.** Tokens run about 28:1 input to output, with research 75% of input (`docs/reports/lr-scaling-cost-provisional-2026-10-14.md`). Prompt caching across turns is broken by a per-second timestamp.
- **The real constraint is RAM, disk and CPU on one host**, as the owner said. A paper-sized computer holds about 5.6 GiB host RSS; Node B has 31 GiB.
- **Policy:** spend is tracked, never capped by the runtime; no new recurring spend (paid APIs, licences, hosts) without owner sign-off.

9.3 Legal and rights

- **Source licences.** Autopaper is commercial (owner ruling 3), so non-commercial sources (CNA, UDN, ransomware.live without approval) are out. Telegram's 2026 terms on AI use need reading before any MTProto connector.
- **Reporting vs copying.** The media research supports the owner's position that transcribing and quoting with attribution is reporting, with edges at substitution, wholesale reproduction, how a source was obtained, and contract terms (`docs/research/autopaper-media-and-rights-2026-10-14/README.md`).
- **EU.** The press publishers' right covers more than very short extracts for two years; the AI Act's Article 50(4) labelling duty for AI-written public-interest text applies from 2 August 2026 unless there is human editorial control and someone holds editorial responsibility. Autopaper publishes with no human step, so labelling is needed for EU readers.
- **Images.** The og:image of a cited article is not Choir's to show; draw maps and charts from cited data, or use free-licensed images.
- **Reputation of real people and institutions.** Scoring public figures' claims raises defamation and data-protection questions. The owner has accepted the defamation risk (ruling of 14 October, about 14:55 UTC) and kept the quality standards as the excellence bar: attribution evidence, several readings, neutrality, right of reply and correction, visible uncertainty. The plan still excludes individual-level tracking in discourse sources. Data-protection law (for EU subjects in particular) is not covered by that ruling and is worth a counsel read before a public view of named people, and before selling one (section 8.6).
- **Erasure vs versioned history.** Dolt keeps history forever, which conflicts with deleting personal data; per-person keys or a rewrite path are needed before personal data enters the versioned graph (`docs/research/puter-everything-2026-10-14/README.md`).

9.4 Product and strategy

- **Order of the two mountains.** The vision says self-development first, then the newspaper; the work this month puts the newspaper's efficiency and scaling first. That is a reasonable owner choice (the paper is the demo and the proof surface), but the self-development gate is the core claim and has been re-opened by the re-engineering.
- **Breadth of the vision vs one builder.** The brief spans an OS, a newspaper, a radio, app agents, a reputation network and a marketplace. The repository has one human author. The stations are sensibly ordered, but Autoradio, app agents and the learning economy are each products in their own right.

- **The research claims are untested.** PSA's hypotheses H1-H5 have not been run; reputation as a moat is "a moat hypothesis" in the deck itself. L3's acceptance (ten hypotheses committed before their evidence, half settled) is the first real data point.
- **Goodharting the ledger.** The metamission lists the failure modes: hypotheses written after the evidence or too vague to fail, padding with near-certain items, counting stories instead of settled commitments, corroboration that measures wire syndication.
- **Process weight.** Panels, problem records and goal files make the work legible and catch real errors, but a station can take four review rounds (LU) and the docs corpus is large (1,290 files). The risk is pace and findability, not correctness.
- **App-store limits.** A mobile app cannot carry the desktop or an app store; Autoradio has to be designed as a focused client of a server-side computer from the start.
- **Onboarding.** Accounts are passkey-only with broken recovery; no second user can be safely onboarded until LA lands.

9.5 Open decisions for the owner (as recorded on 14 October)

1. The Go module route for capsules (A, B or C); B (host module cache) is built and deployed.
2. LU's open questions (O1-O3b, O5, O7-O10) and the LU1b drain decision.
3. SP0: which host, recovery targets, and offline copies of the escrow custodian key and platform signing seed (the latter copied 14 Oct).
4. The 12-hour maximum age for memory snapshots.
5. The reader spec decision.
6. Auth before self-development; which extra sign-in methods.
7. Whether LT's first targets include the live paper's host services or only forks.
8. Paid search accounts; Cofacts application (owner said "in a couple of days").
9. Whether to keep Autoradio after L2-L4 or pull a demo forward.

Appendix A. Glossary

Term	Meaning
Autopaper	The automatic newspaper: sources polled 24/7, stories synthesized by desks, published automatically. Formerly "World Wire", "Universal Wire" (retired names; code still says `universal-wire` in places)
Autopaper object graph	The shared graph of published objects (captures, published stories, later op-eds and templates) on the host, which computers adopt onto their own tapes. Physically Store B behind corpusd
Autoputer	The implementation name of the guest daemon that runs inside each computer's VM. Not the product name; the product object is the persistent computer
Autoradio	The planned mobile listening and voice surface (vision)
Box score	A view over settled records for one scope (speaker, outlet, organization, agent), with sample sizes and uncertainty; never a stored or single trust number (designed)
Capsule	A disposable, guest-local sandbox where engineering makes changes; its output is a frozen effect bundle
Capsule effect bundle	Frozen, content-addressed proposed change bound to a computer and event head: the only self-development candidate
Claim	One attributed assertion (speaker S said Y at time T, anchored to a quote): the precommitment Choir records on a speaker's behalf (designed)
Capture	One fetched source item (`choir.web_capture`), with its body stored by hash
Cast, Ask, Note, Report, Resolve	Inter-desk acts, each a precommitment record
corpusd	Host service fronting Store A and Store B: event heads, publications, captures, escrow, checkpoints
Desk	A persistent root agent with a role (Texture, management, engineering, research) running Go cells
Dolt	A SQL database with Git-like versioning, used for the stores
Gate 1 / Gate 2 / Gate 3	The 9 October roadmap: stable computer; supervised self-development; Autopaper
Heresy	A named flaw in code, docs, tests, prompts or process; counted as discovered, introduced, repaired
Jev	A fast, cheap typed-judgement model (citation support, intake labels, importance)
Merge recall	The share of same-event pairs from different outlets that ended in the same story (L2a's quality measure)
Mutation class	green, yellow, orange, red, black: the risk class of a change, which sets its ceremony
Node B / node-a	The single production-staging host / a shared host used as a Nix builder
OSINT	Open-source intelligence: knowledge built from public sources
Planned-restart marker	A durable marker written before an update restart; its presence lets the next boot resume work once
Precommitment record	A typed prediction committed before the evidence that resolves it, later settled and scored
PSA	Prospective self-alignment, the research theory around precommitment records
Realization	A running VM instance of a computer; replaceable without changing the computer's identity
Say-do alignment	The share of an actor's checkable commitments its observed actions kept: "can I take you at your word?" (designed)
RLM	Recursive language model: a model loop that writes code cells and can delegate to sub-agents
Staging	choir.news, the acceptance environment
Station	A unit of a metamission with its own goal file, acceptance and rollback
Store A / Store B	Host Dolt servers: platform control / shared corpus and object graph
Tape	The canonical per-computer event chain; the authority from which state is projected
Texture	Both the document type (a living, versioned document) and the desk that is its sole agent writer
Throughline	The current executable goal-file format for missions
Tracer	The deployed end-to-end probe that mints a test computer and records a trace bundle

Term	Meaning
Yaegi	A Go interpreter used to run desk cells

Appendix B. Station index (as of 14 October 2026)

Station	Name	Status	Goal file or source
L0	Close the re-engineering metamission (E3, A1, C1e, final review)	working	`docs/definitions/choir-reeng-metamission-2026-10-12.md`
L1	No crash loops (L1a landed; L1b, L1c next)	working	`docs/definitions/choir-ledger-l1-restart-2026-10-13.md`
L2	Bounded attention (L2b: digest, triage, burst)	pending	planned path `docs/definitions/choir-ledger-l2-bounded-attention-2026-10-13.md` (not yet authored)
L2a	Synthesis intake: one story per event across languages	working	`docs/definitions/choir-ledger-l2a-synthesis-intake-2026-10-14.md`
LR	Resource efficiency and model cost (with S3, SA)	working	`docs/definitions/choir-ledger-lr-resource-2026-10-13.md`
LU	Guest updates: pull with a bounded drain	working (reviewed)	`docs/definitions/choir-ledger-lu-guest-updates-2026-10-14.md`
L3	Hypotheses settled and scored	pending (intent)	planned path `docs/definitions/choir-ledger-l3-hypotheses-settled-2026-10-13.md` (not yet authored)
L4	Taiwan and East Asia vertical	pending (intent)	planned path `docs/definitions/choir-ledger-l4-taiwan-2026-10-13.md` (not yet authored)
L5	Evidence reader and Autoradio prototype	pending (intent)	planned path `docs/definitions/choir-ledger-l5-read-models-2026-10-13.md` (not yet authored)
LP	App packages (S6 commit gate, then S7)	pending	`docs/definitions/choir-appdev-s7-app-packages-2026-10-01.md`
LA	Auth: passkeys plus email magic link, recovery	pending (unauthored)	`docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`
SP	Production infrastructure: second host, off-host copies, failover	deferred	`docs/definitions/choir-appdev-sp-production-infrastructure-2026-10-09.md`
LT	Agentic red teaming	pending (unauthored)	`docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`
LF	Front-page ranking and personalization (LF0, LF1 live)	pending (intent)	`docs/reports/autopaper-ranking-and-contributions-intent-2026-10-14.md`
LC	Contributions, op-eds, editing	pending (intent)	same
S2a	Management delegation to sub-RLMs	deployed 14 Oct	`docs/reports/management-delegation-design-2026-10-14.md`
S3	Snapshot hibernation	deployed, proven	`docs/reports/s3-snapshot-hibernate-design-2026-10-14.md`
S4-S6	Capsule open world, live preview, commit gate (Gate 2)	pending	`docs/definitions/choir-appdev-s6-commit-gate-full-release-2026-10-01.md`
S8-S11	Publication and adoption, forks and fleets, org templates, fleet pushes	later	supervised app-development metamission

Appendix C. Key file map

Topic	Path
Operating contract	`CLAUDE.md`, `AGENTS.md`
Vision	`docs/choir-vision.md`, `README.md`
Doctrine	`docs/choir-doctrine.md`, `docs/agent-product-doctrine.md`
Computer ontology and store taxonomy	`docs/computer-ontology.md`
Foundational argument	`docs/signal-is-sparse-not-the-learner-2026-08-01.md`
Precommitment records	`docs/Precommitment Records — Engineering Memo.md`, `docs/prospective-self-alignment-2026-10-09.md`
Autopaper design	`docs/autopaper-sw-design-2026-10-09.md`
Current work view	`docs/ACTIVE.md`, `docs/mission-graph.yaml`
Active goal	`docs/definitions/choir-ledger-metamission-2026-10-13.md`
Owner rulings and sequencing	`docs/reports/owner-roadmap-rulings-2026-10-14.md`, `docs/reports/roadmap-sequencing-appdev-security-2026-10-14.md`
App agents research	`docs/research/puter-everything-2026-10-14/README.md`
Media and rights research	`docs/research/autopaper-media-and-rights-2026-10-14/README.md`
Pre-flight questions	`docs/standing-questions.md`
Claims, events, entities, box scores	`docs/reports/claims-events-entities-ontology-2026-10-14.md`
Search efficiency and reuse	`docs/reports/search-efficiency-and-reuse-2026-10-14.md`
Local embeddings	`docs/research/embeddinggemma-2-cpu-2026-10-14.md`
VM density limits	`docs/reports/vm-density-limits-2026-10-14.md`
Provider credentials (platform only)	`internal/provider/provider.go` (`ChatGPTProvider`, Codex OAuth)
Guest runtime and reducer	`internal/agentcore/`, `internal/agentcore/rlm_reduce.go`
Desk verbs	`internal/yaegikernel/choir.go`
Texture writer	`internal/textureowner/texture_cell_author.go`, `internal/store/texture_turn.go`
Commitment records	`internal/types/commitment.go`, `internal/store/inbox.go`
Event chain	`internal/computerevent/`, `internal/platform/computer_events.go`
Object graph	`internal/objectgraph/`, `internal/platform/objectgraph_store.go`
Sources and schedules	`internal/sources/registry.json`, `internal/agentcore/source_schedule.go`
Publication and front page	`internal/platform/service.go`, `internal/platform/publication_front.go`, `frontend/src/lib/autopaper.ts`
Gateway, search, Jev	`internal/gateway/`, `internal/gateway/searchplane/`, `internal/gateway/jev.go`
Costs	`internal/llmcost/`
VM lifecycle	`internal/vmctl/`, `internal/vmmanager/`
Capsules and self-development	`internal/capsule/`, `internal/selfdev/operations.go`
Auth	`internal/auth/`, `cmd/auth/main.go`
CLI	`cmd/choir/main.go`
Host and guest config	`flake.nix`, `nix/node-b.nix`, `nix/autoputer-vm.nix`
CI and deploy	`.github/workflows/ci.yml`, `.github/workflows/race.yml`
Tracer and probes	`scripts/reeng_tracer.mjs`, `scripts/l2a_window_pairs.py`

Appendix D. How this report was made

Read-only research on 14 October 2026 between about 13:50 and 15:40 UTC (section 8 and the folded-in designs added after 15:05 UTC): the documents named above; `git log origin/main` from 7 October (midnight US Eastern) to 7173cc4a0; a line count of tracked files; a code survey of the packages named; and two public reads of choir.news (`/health` and `/api/platform/publications/front` at 14:20 UTC). No service, staging computer or host was touched. Commit counts by area come from subject-line grouping and are approximate. Where the report states a hypothesis rather than a traced finding, it says so.